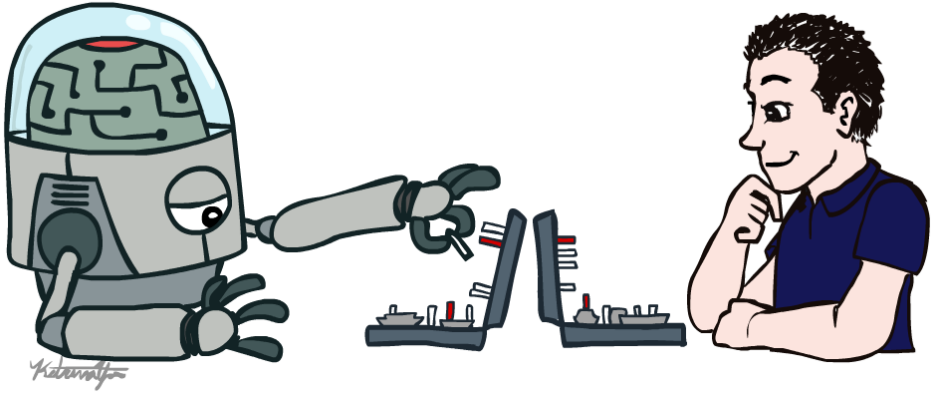
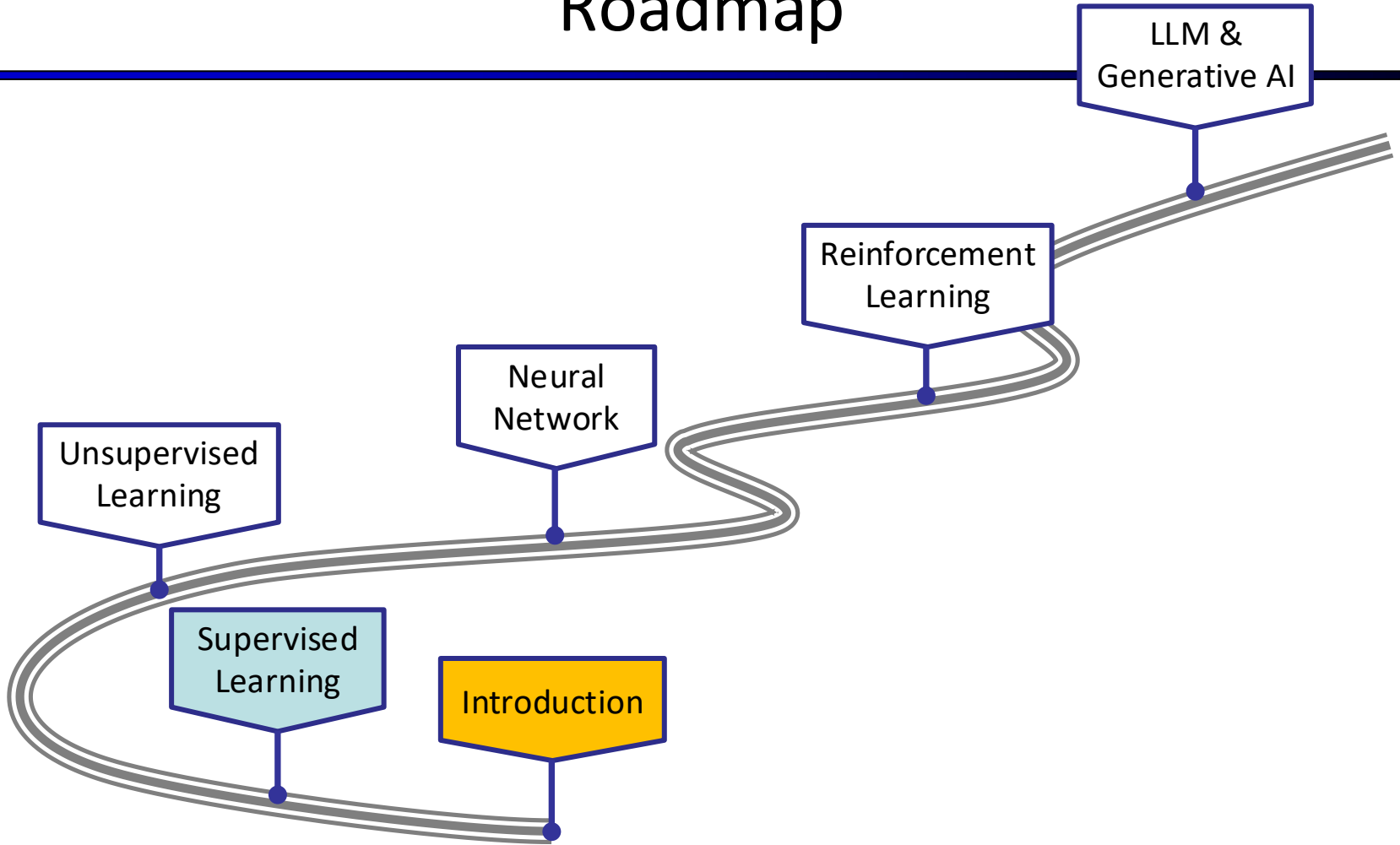
Artificial Intelligence

Supervised Learning



Yan Lyu

Roadmap


LLM &
Generative AI
Reinforcement
Learning
Neural
Network
Unsupervised
Learning
Supervised
Learning
Introduction

Textbook

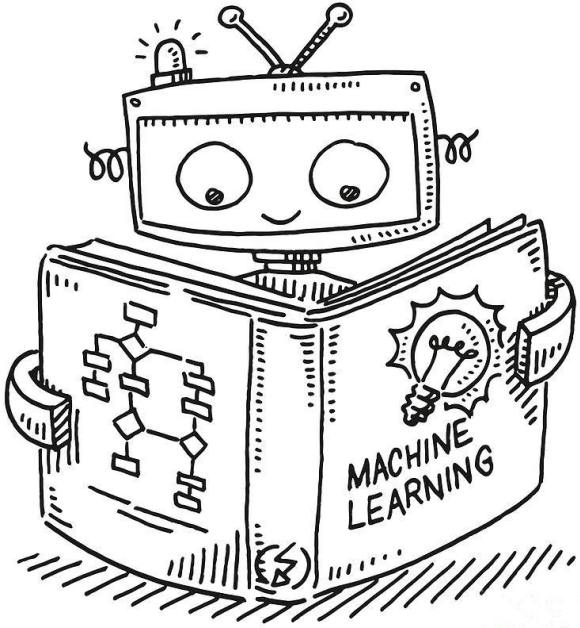
1. **[主教材]** 阿斯顿·张, 李沐 等. **《动手学深度学习》（PyTorch版）**. 人民邮电出版社, 2023. (配套在线资源: https://zh.d2l.ai/)
2. **[参考书-大模型]** Sebastian Raschka. **《Build a Large Language Model (From Scratch)》**. Manning Publications, 2024.
3. **[参考书-强化学习]** Richard S. Sutton. **《强化学习（第2版）》**. 电子工业出版社, 2019.
4. **[前沿技术文档]**
    1. Dan Jurafsky. *Speech and Language Processing (3rd ed. draft)*. (针对LLM理论)
    2. *Hugging Face / OpenAI / LangChain 官方技术文档*. (针对RAG与Agent开发实验)

Course Information

- 作业:
  - Project 1：经典机器学习：端到端的数据挖掘与预测 （25%，1-4周）
  - Project 2：深度学习与视觉：基于 PyTorch 的图像识别 （25%，5-8周）
  - Project 3：强化学习：训练你的专属游戏 AI Agent （25%，8-12周）
  - Project 4：生成式 AI 时代：构建本地私有化 RAG 智能问答助手 （25%，12-16周）
- 考核:
  - 期末汇报：随机抽取四个项目中任意一个项目进行期末汇报。

Today

- **Regression: Linear Regression**
- Classification: Logistic Regression
- Discussion



How much is my house worth?

Look at recent sales in my neighborhood

- How much did they sell for?

Data

$(x_1 = 110\ m^2,\ y_1 = 152{,}000\ \$)$

$(x_2 = 240\ m^2,\ y_2 = 278{,}000\ \$)$

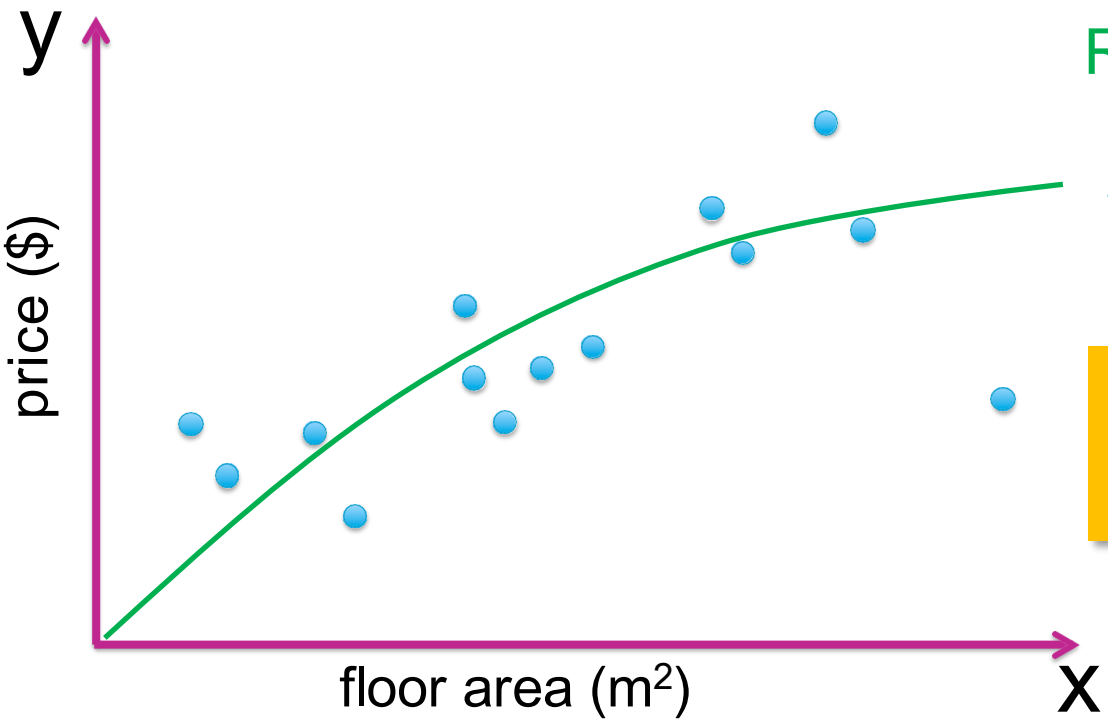
$(x_3 = 150\ m^2,\ y_3 = 160{,}000\ \$)$

$(x_4 = 173\ m^2,\ y_4 = 183{,}000\ \$)$

$(x_5 = 210\ m^2,\ y_5 = 223{,}000\ \$)$

$\vdots$

Model – How we *assume* the world works



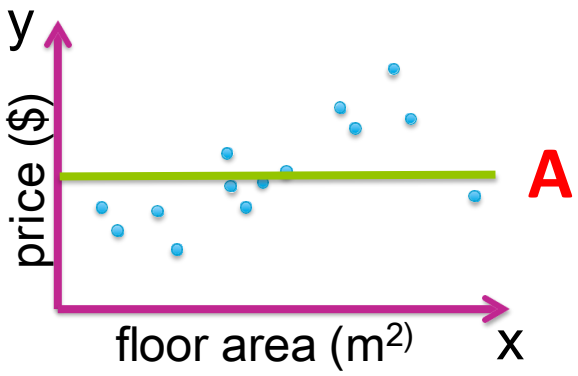
Regression model

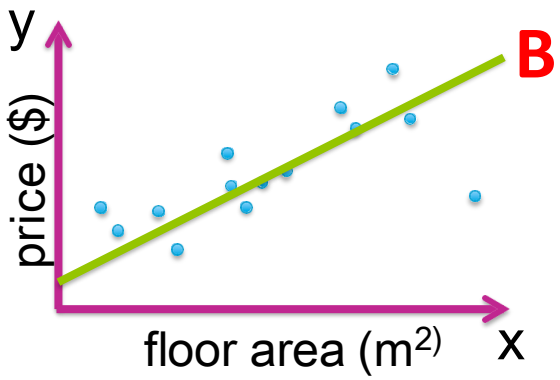
$$y_i = f(x_i) + e_i$$

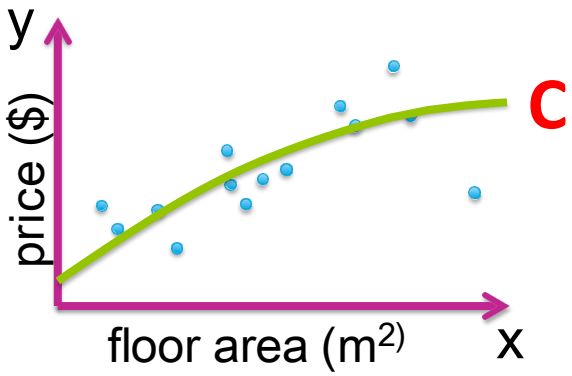
"Essentially, all models are wrong, but some are useful."
George Box, 1987.

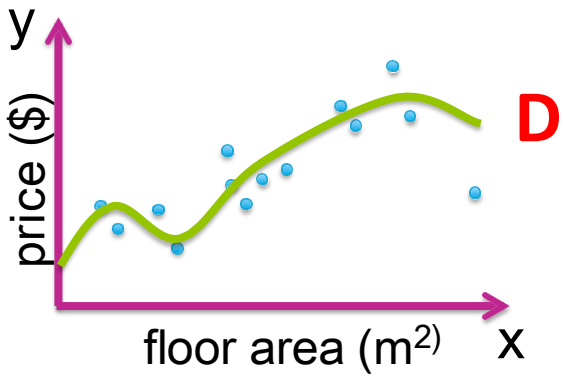
https://en.wikipedia.org/wiki/George_E._P._Box
https://en.wikipedia.org/wiki/All_models_are_wrong

Which model f(x)?

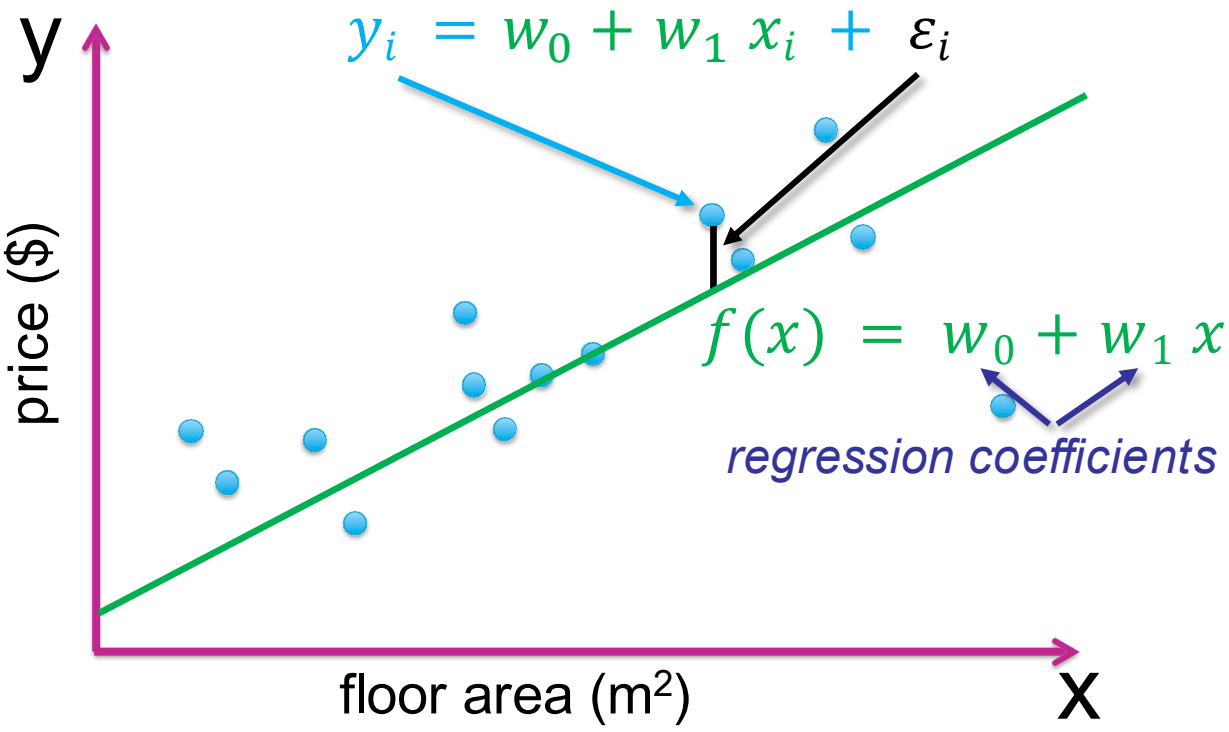






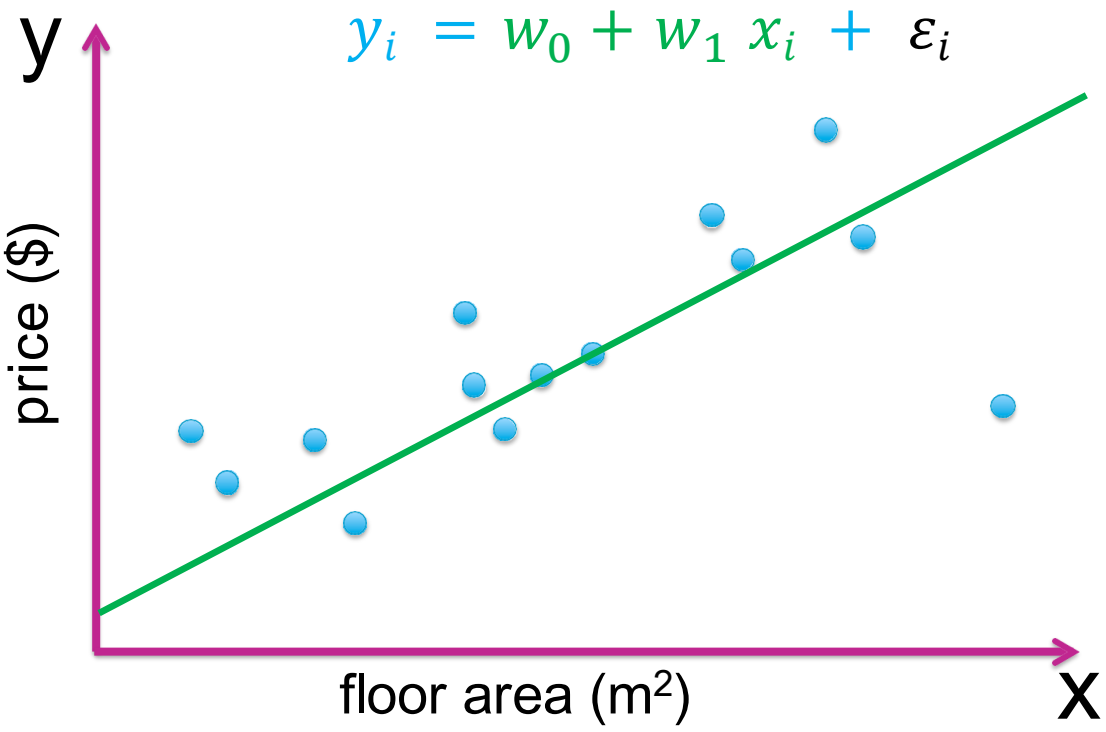


Simple linear regression model



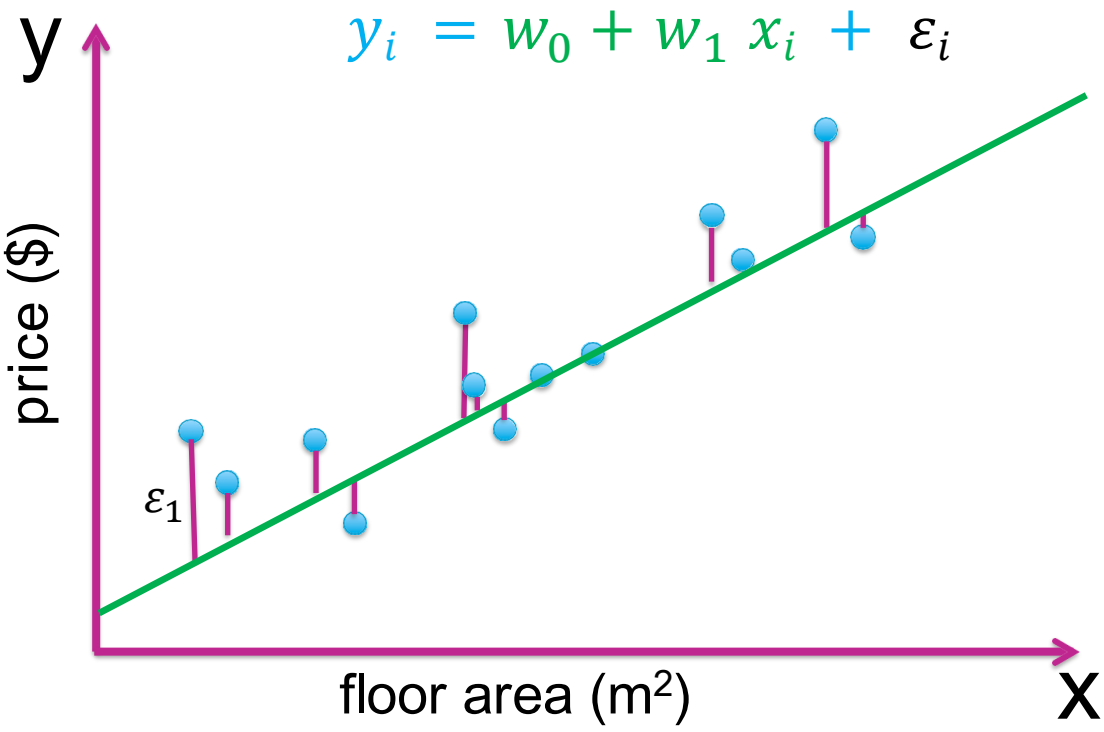
Simple linear regression model

$$y_i = w_0 + w_1 x_i + \varepsilon_i$$



"Cost" of using a given line

$$y_i = w_0 + w_1 x_i + \varepsilon_i$$



$$\varepsilon_1 = y_1 - (w_0 + w_1 x_1)$$

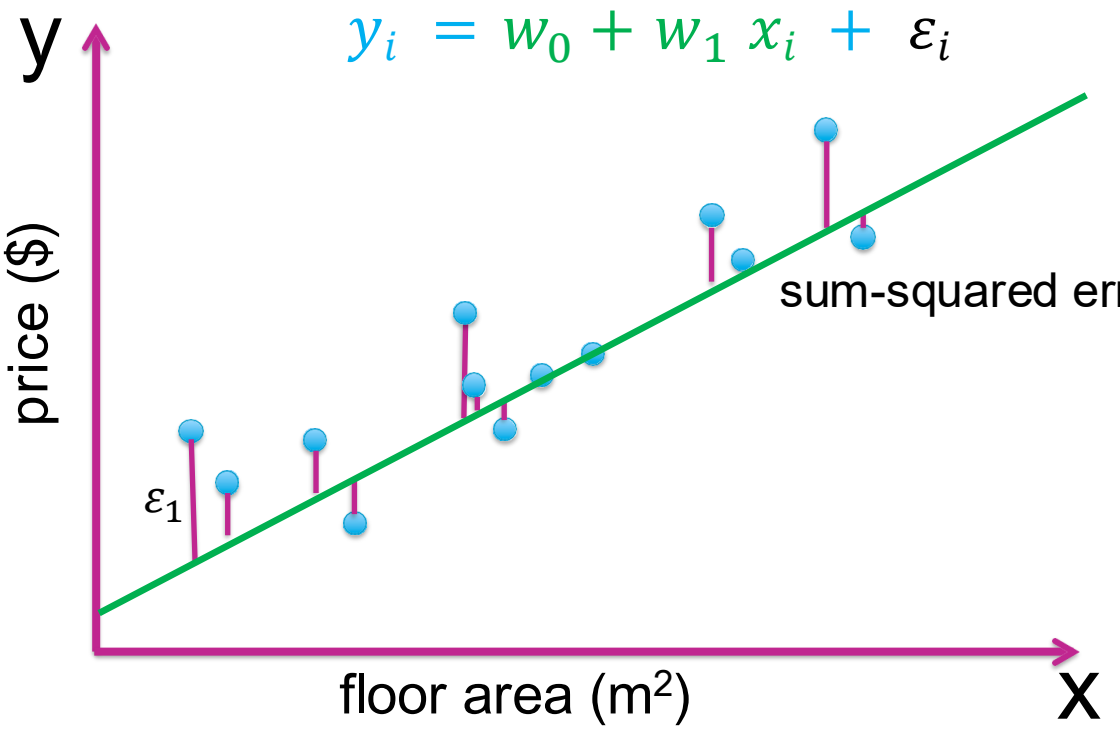
$$\varepsilon_2 = y_2 - (w_0 + w_1 x_2)$$

$$\vdots$$

$$\varepsilon_n = y_n - (w_0 + w_1 x_n)$$

"Cost" of using a given line

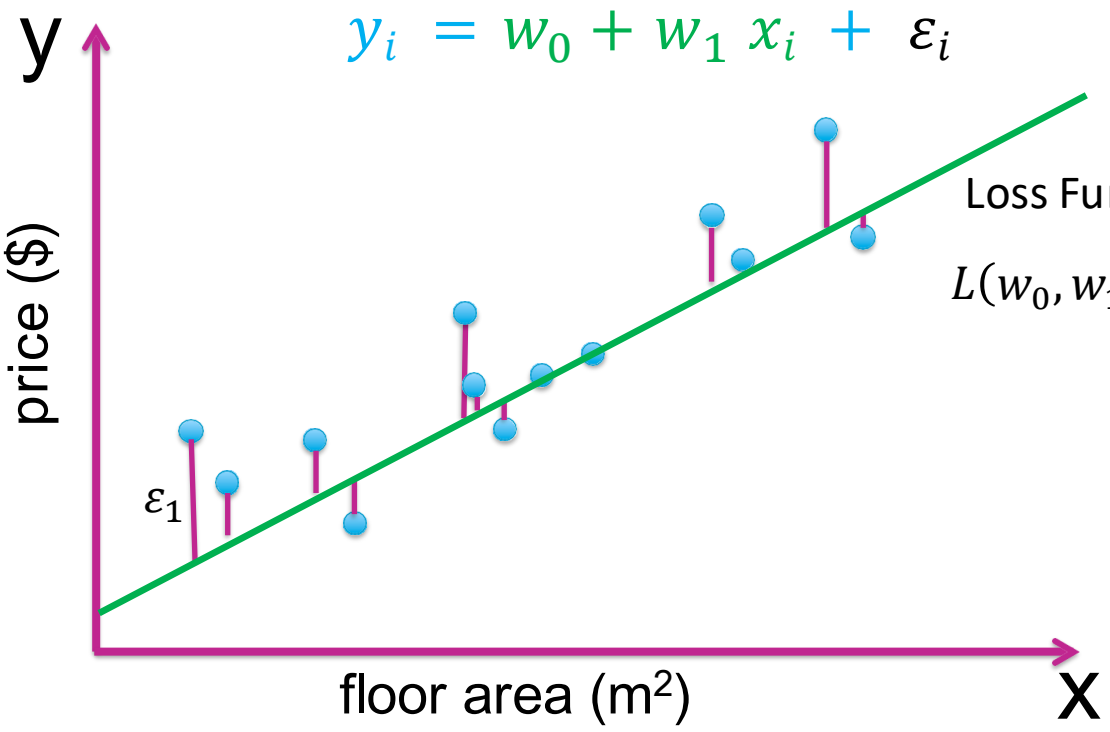
$$y_i = w_0 + w_1 x_i + \varepsilon_i$$



$$\text{sum-squared error} = \left(y_1 - (w_0 + w_1 x_1)\right)^2 + \left(y_2 - (w_0 + w_1 x_2)\right)^2 \vdots + \left(y_n - (w_0 + w_1 x_n)\right)^2$$

"Cost" of using a given line

$$y_i = w_0 + w_1 x_i + \varepsilon_i$$

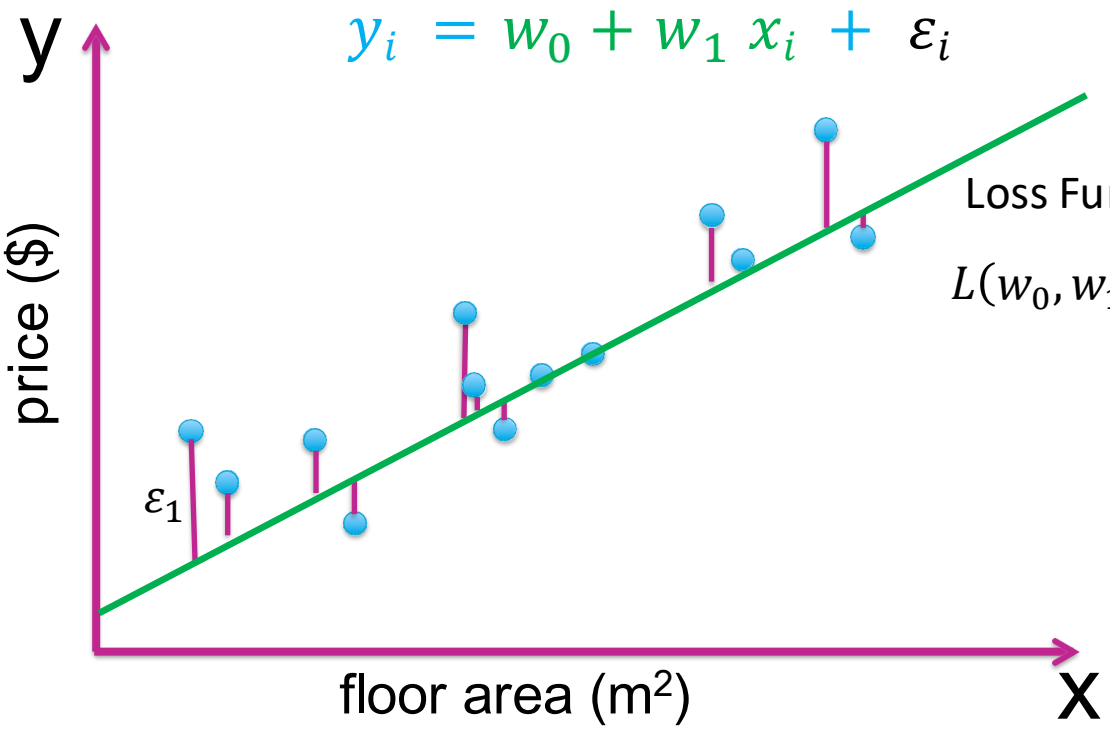


Loss Function:

$$L(w_0, w_1) = \left(y_1 - (w_0 + w_1 x_1)\right)^2 + \left(y_2 - (w_0 + w_1 x_2)\right)^2 \vdots + \left(y_n - (w_0 + w_1 x_n)\right)^2$$

"Cost" of using a given line

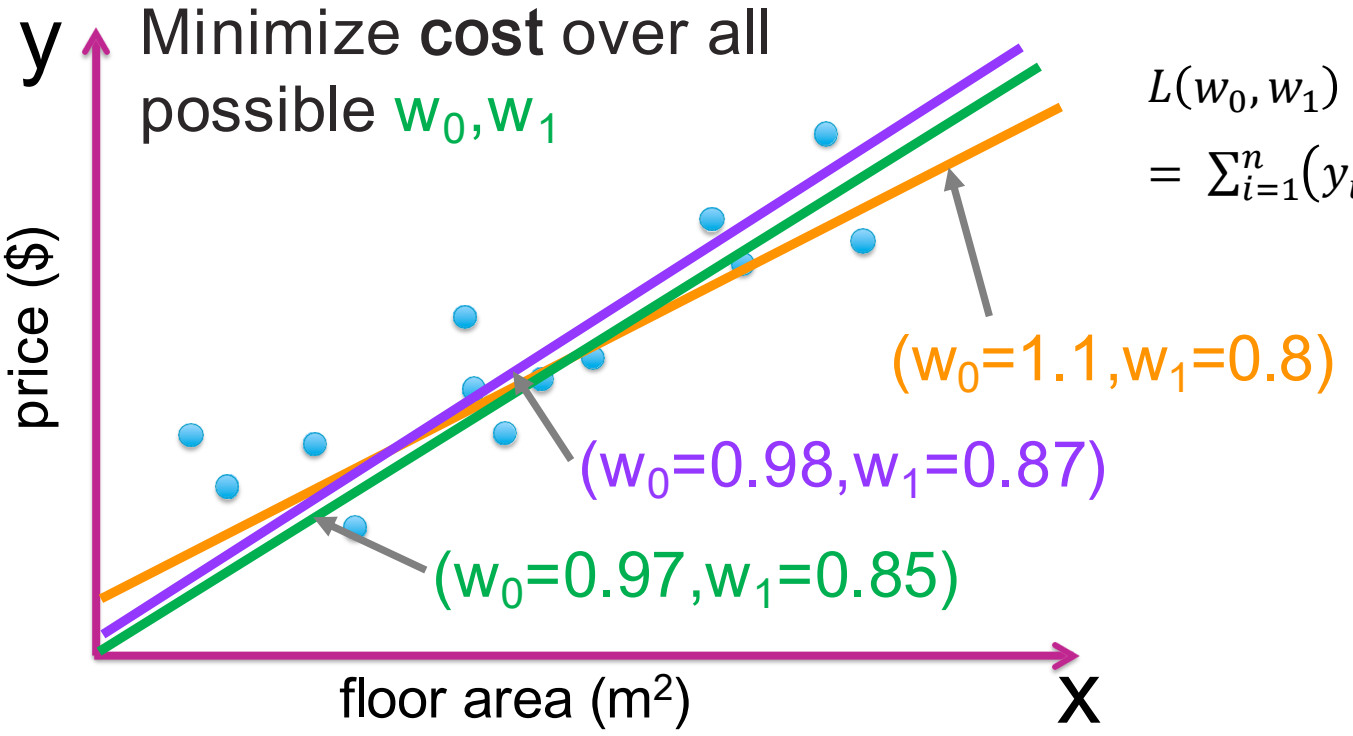
$$y_i = w_0 + w_1 x_i + \varepsilon_i$$



Loss Function:

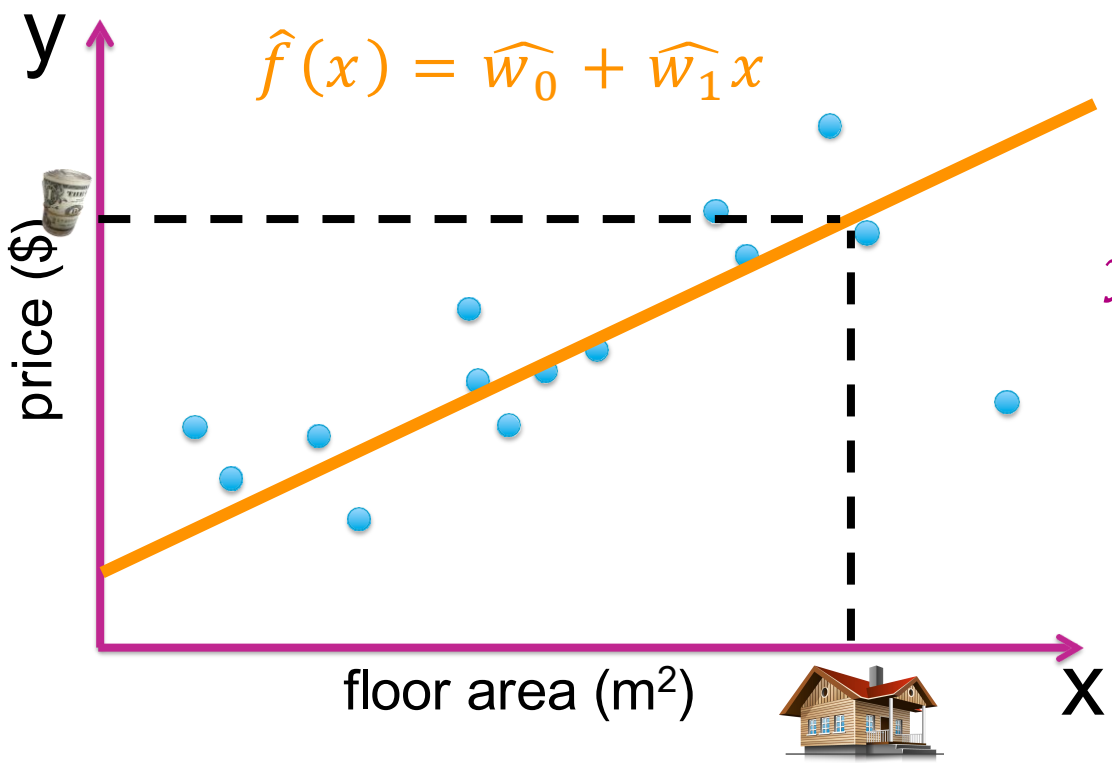
$$L(w_0, w_1) = \sum_{i=1}^{n} \left(y_i - (w_0 + w_1 x_i)\right)^2$$

Find "best" line



$$L(w_0, w_1) = \sum_{i=1}^{n} \left(y_i - (w_0 + w_1 x_i)\right)^2$$

Predicting your house price



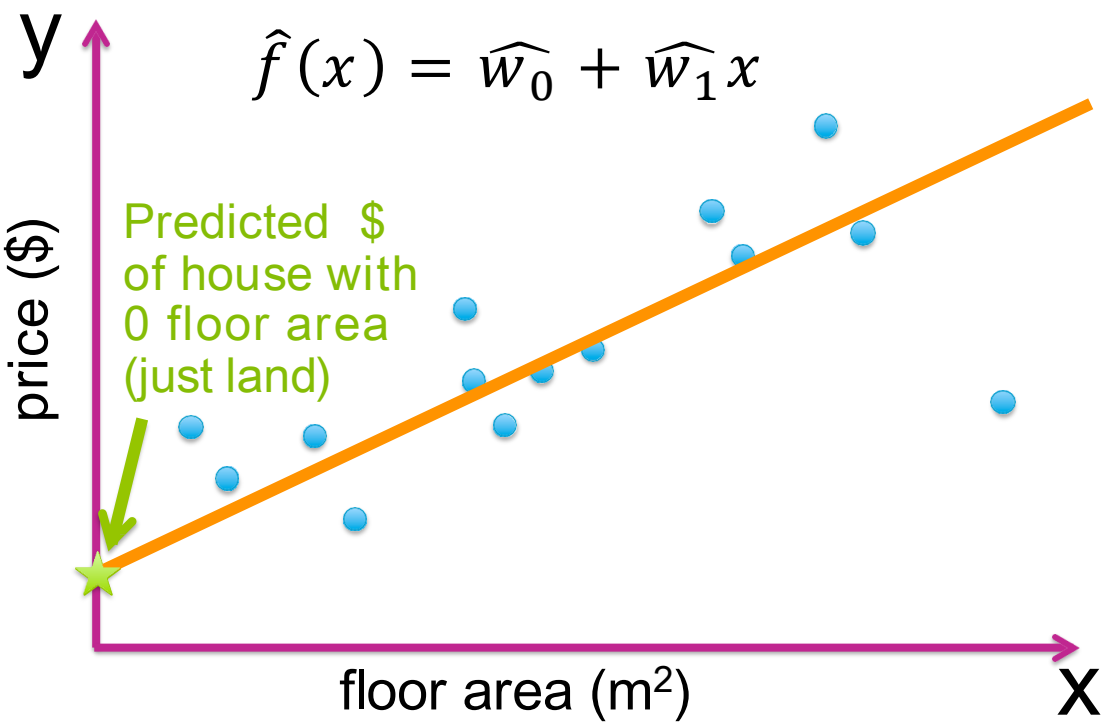
Regression model:

$$y_i = w_0 + w_1 x_i + \varepsilon_i$$

Best guess of your house price:

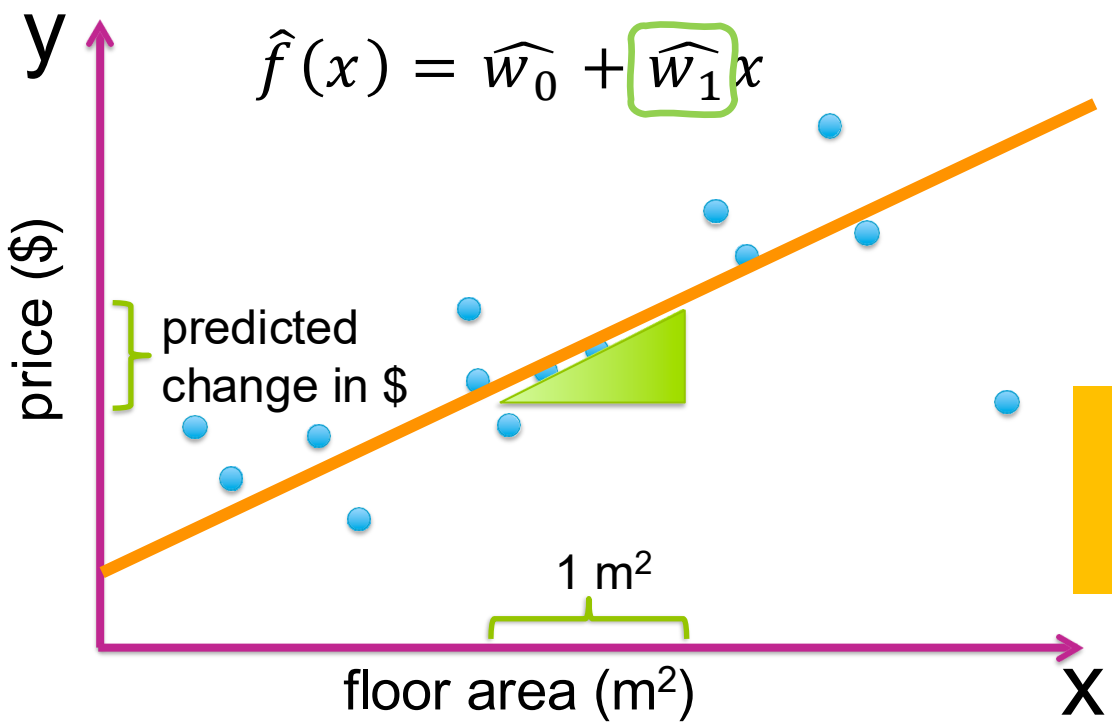
$$\hat{y} = \widehat{w_0} + \widehat{w_1} x$$

Interpreting the coefficients



$$\hat{f}(x) = \widehat{w_0} + \widehat{w_1}x$$

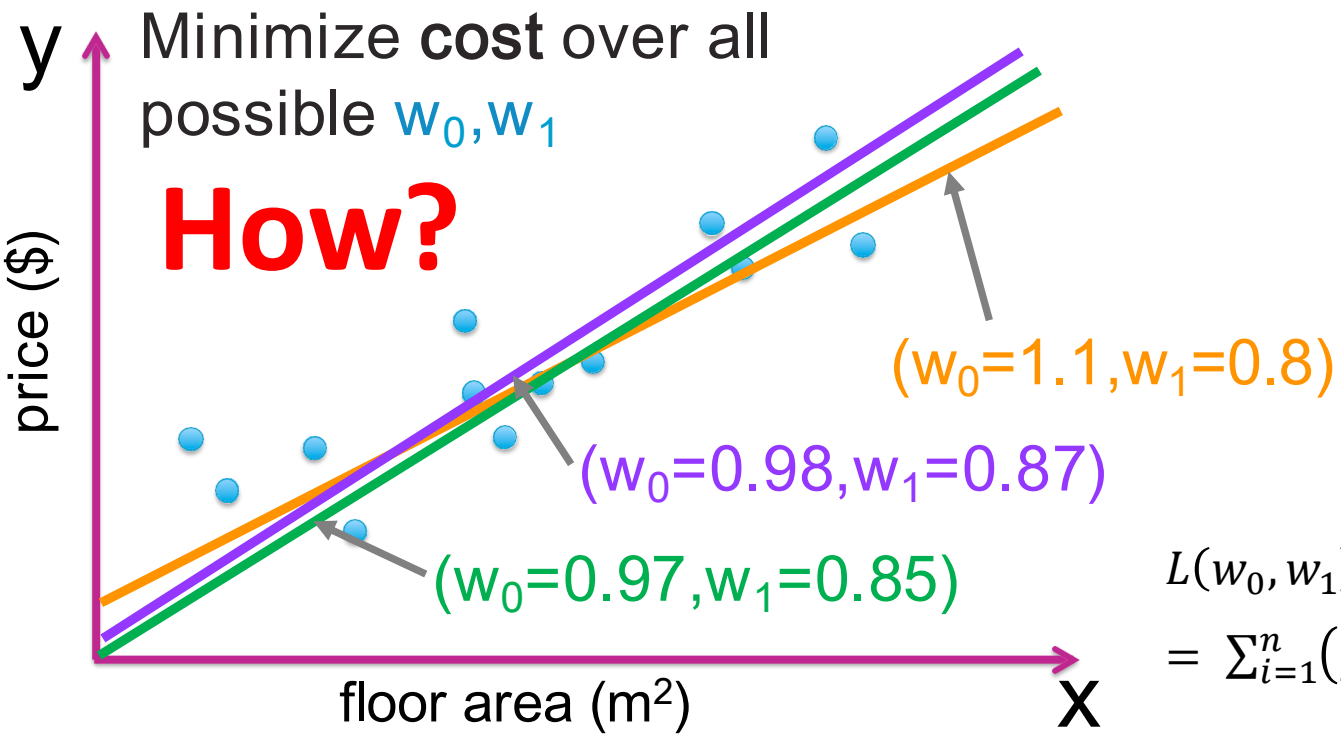
Interpreting the coefficients



$$\hat{f}(x) = \widehat{w_0} + \boxed{\widehat{w_1}}\,x$$

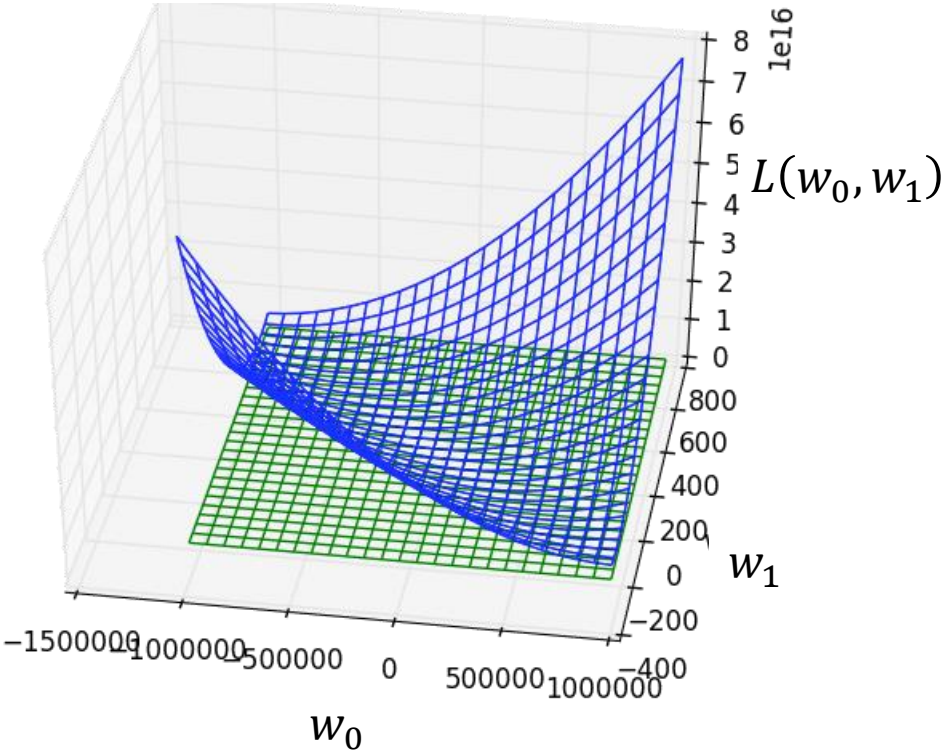
Warning: magnitude depends on units of both features and observations

Algorithms for fitting the model



$$L(w_0, w_1) = \sum_{i=1}^{n} \left(y_i - (w_0 + w_1 x_i)\right)^2$$

Minimizing the cost



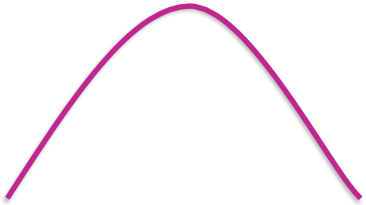
Minimize function over all possible w_0, w_1

$$\min_{w_0, w_1} \sum_{i=1}^{n} \left(y_i - (w_0 + w_1 x_i)\right)^2$$

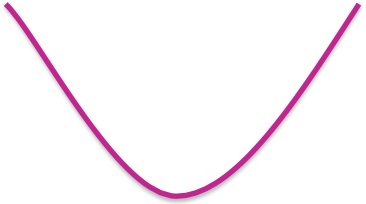
An aside on optimization

Convex/concave functions

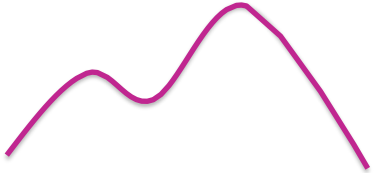
CONCAVE

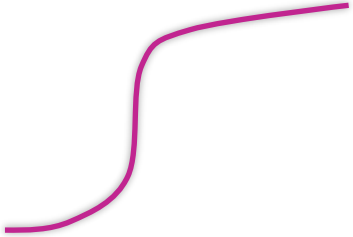


CONVEX

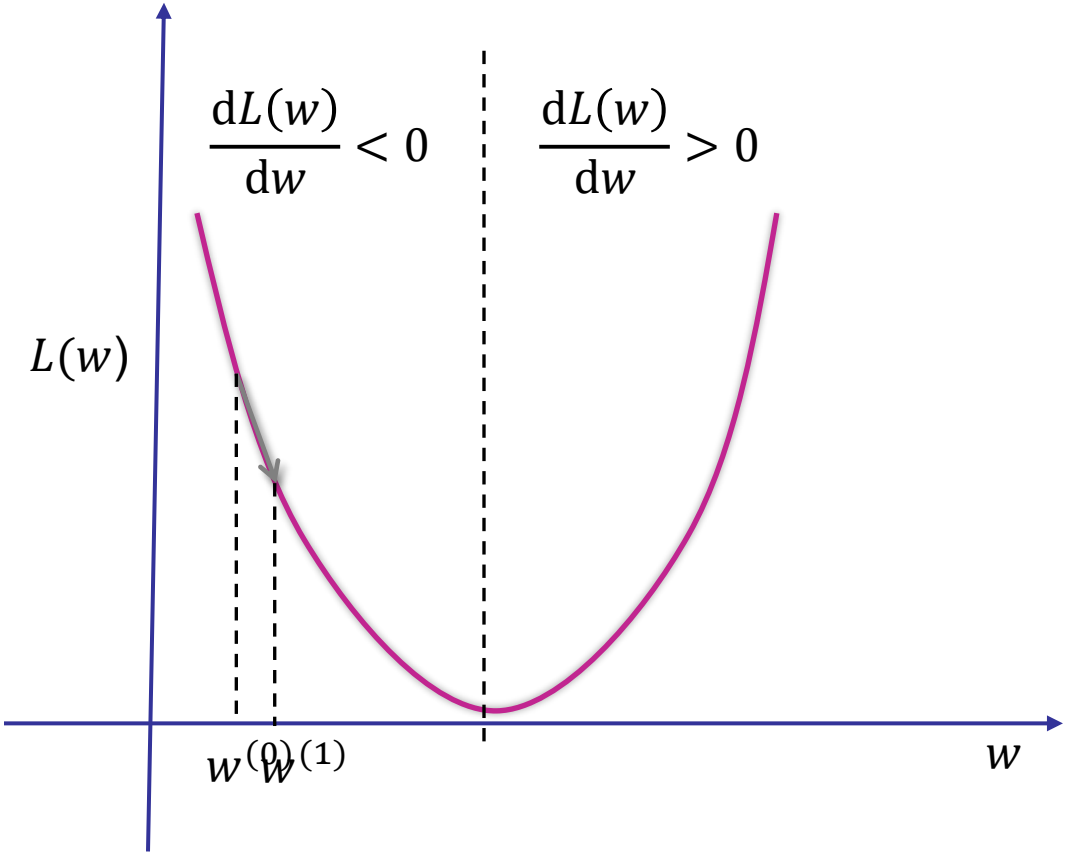


NEITHER



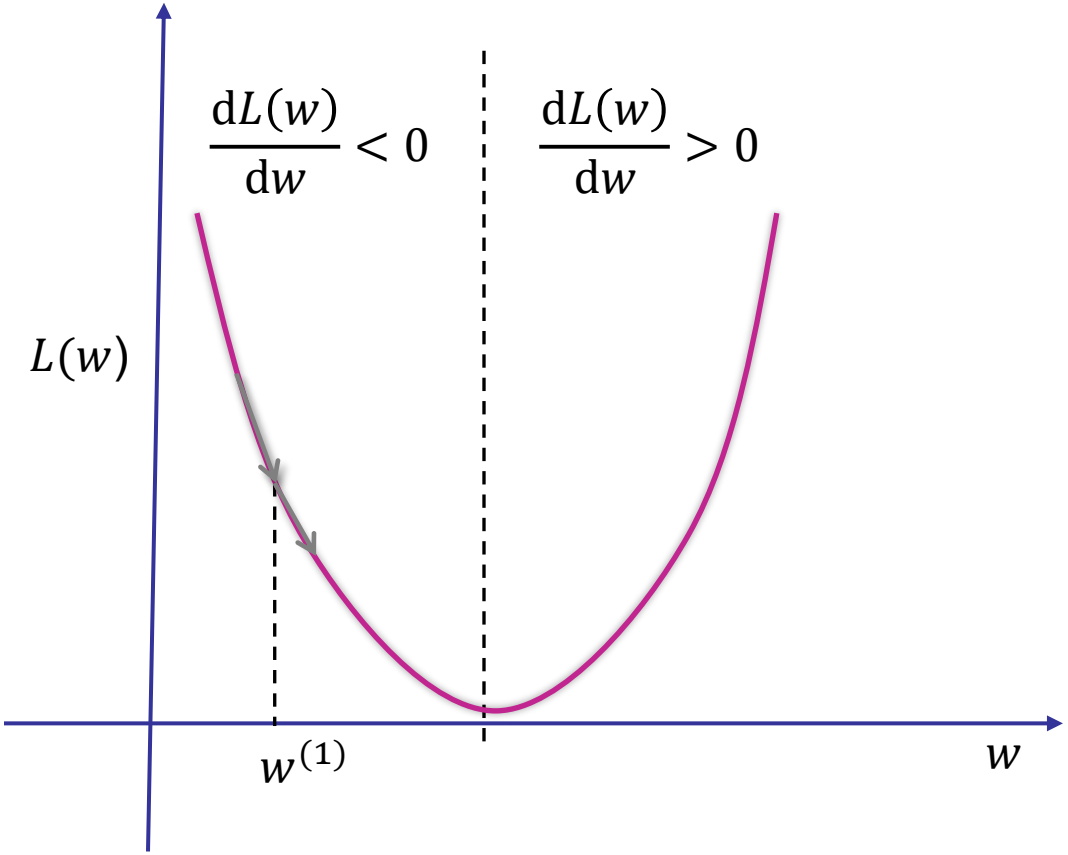


Gradient Descent



$$w^{(t+1)} \leftarrow w^{(t)} - \eta \frac{dL(w)}{dw}$$

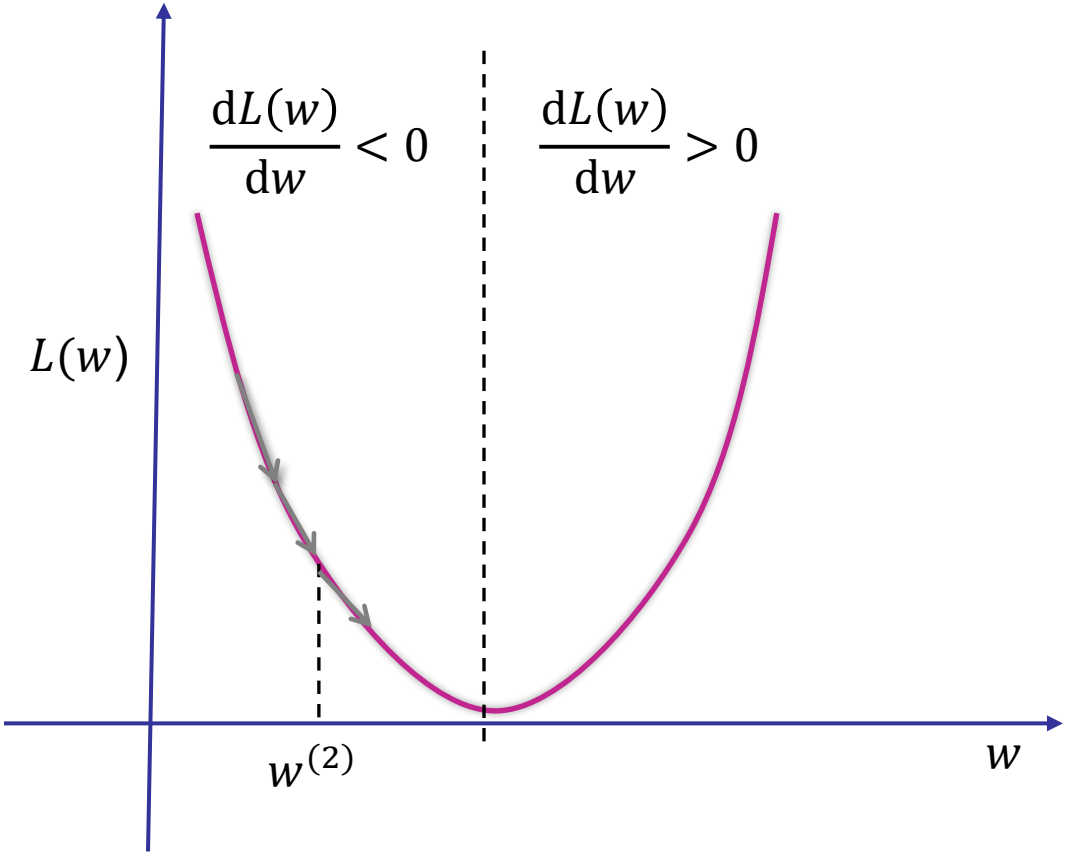
Gradient Descent



$$w^{(t+1)} \leftarrow w^{(t)} - \eta \frac{dL(w)}{dw}$$

$$t \leftarrow t + 1$$

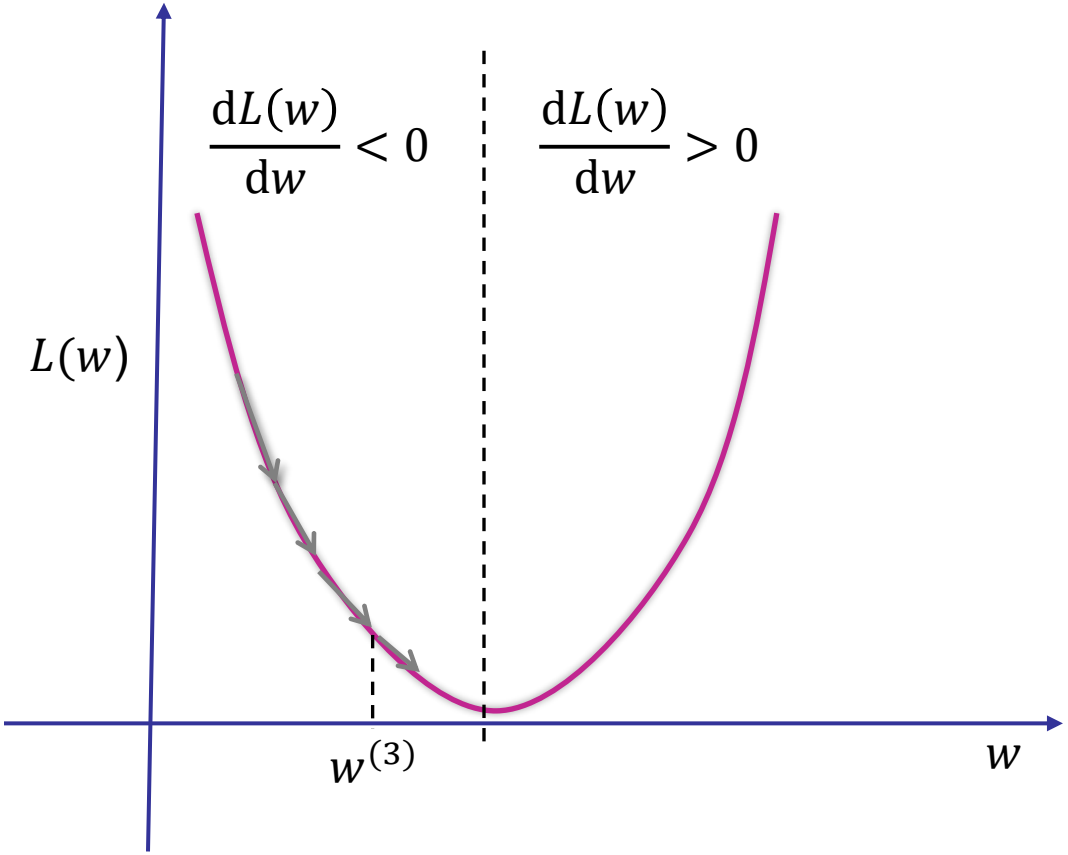
Gradient Descent



$$w^{(t+1)} \leftarrow w^{(t)} - \eta \frac{dL(w)}{dw}$$

$$t \leftarrow t + 1$$

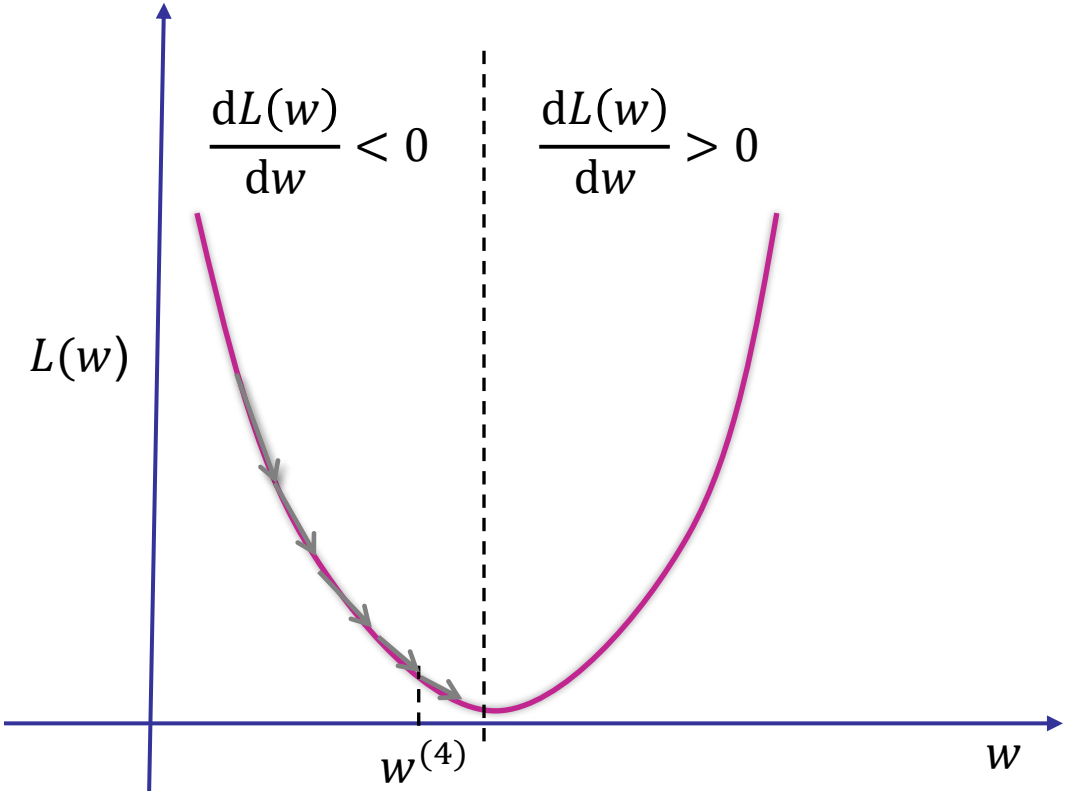
Gradient Descent



$$w^{(t+1)} \leftarrow w^{(t)} - \eta \frac{dL(w)}{dw}$$

$$t \leftarrow t + 1$$

Gradient Descent



Convergence Criteria: $\left|\frac{dL(w)}{dw}\right| < \epsilon$

Algorithm:

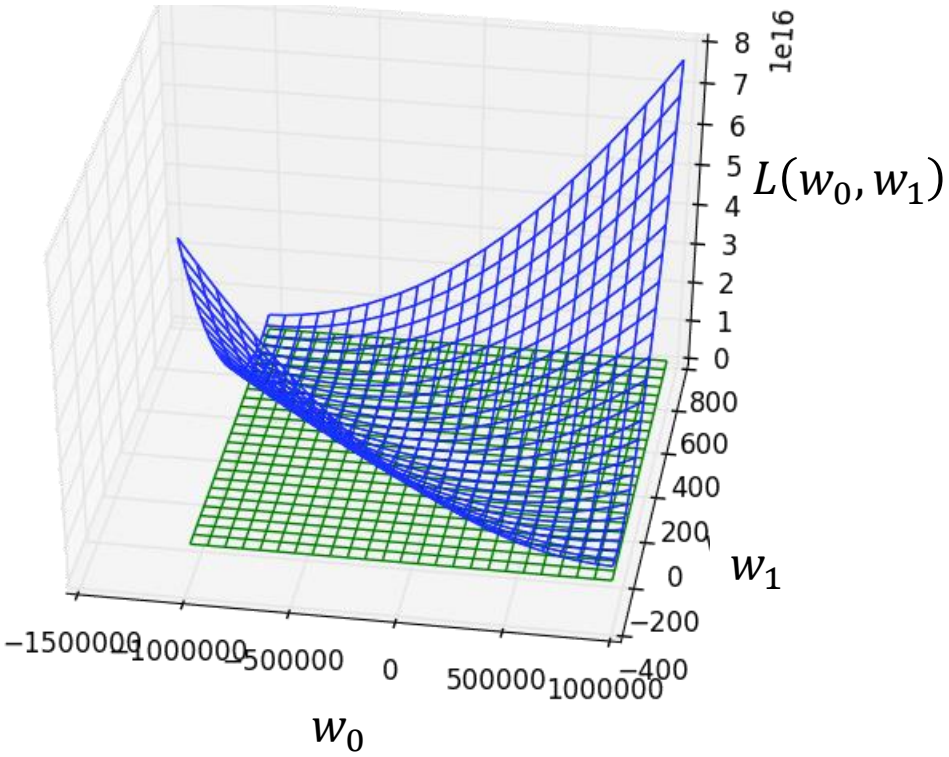
while not converged

$$w^{(t+1)} \leftarrow w^{(t)} - \eta \frac{dL(w)}{dw}$$

$$t \leftarrow t + 1$$

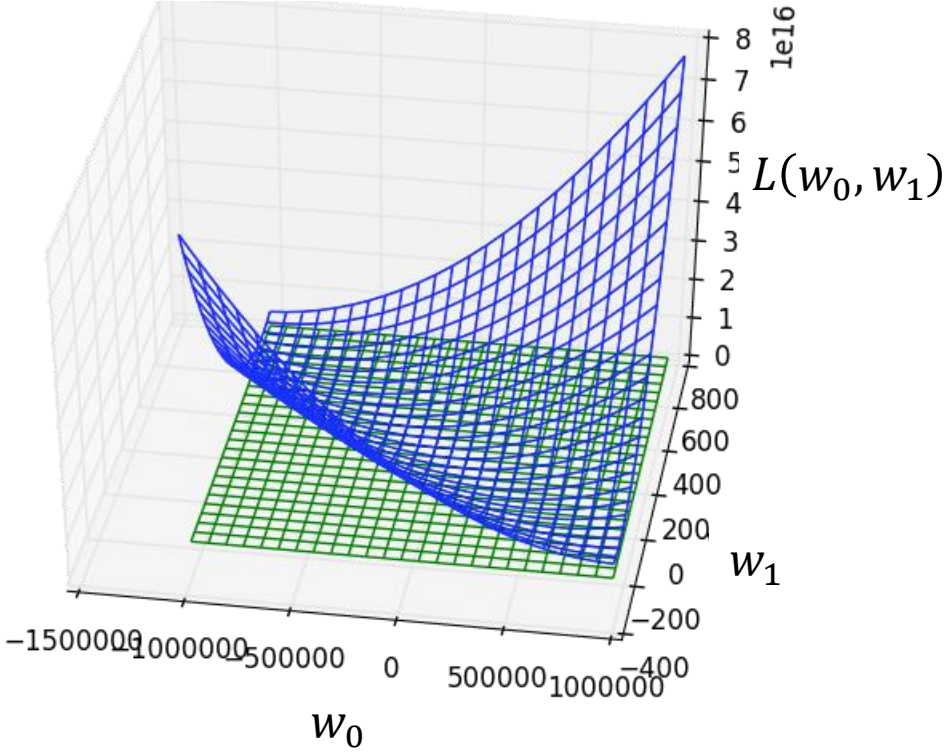
$$\eta_t = \alpha / t$$

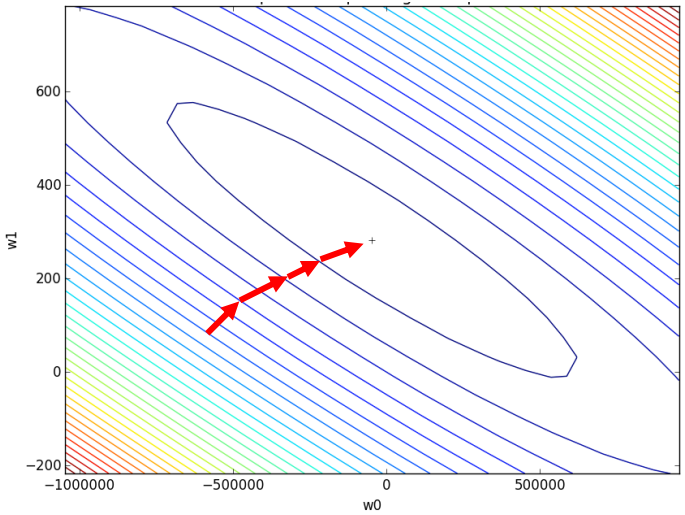
Multiple dimensions: Gradients Descent



$$\nabla L(\boldsymbol{w}) = \begin{bmatrix} \frac{\partial L}{\partial w_1} \\ \frac{\partial L}{\partial w_2} \\ \frac{\partial L}{\partial w_3} \\ \vdots \end{bmatrix}$$

Contour plots





Why Gradient Descent?

- The beauty of GD is that it works for many types of models and loss function as long as you can take the gradient.
  - Usually, taking gradient is much easier than solving it!
  - Also, GD often finds solution faster than analytic solution.
- Modeling recipe:
  - Pick model
  - Pick loss function
  - Fit model by running gradient descent

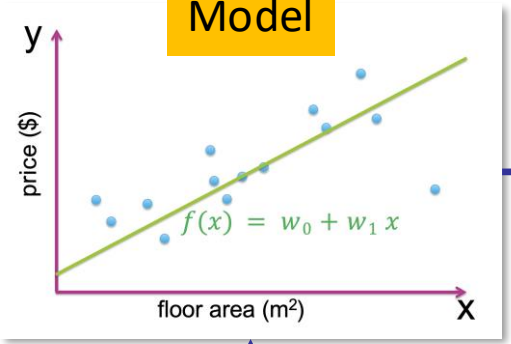
Linear Regression

Training Data

(x_1 = 110 m², y_1 = 152,000 \$)
(x_2 = 240 m², y_2 = 278,000 \$)
(x_3 = 150 m², y_3 = 160,000 \$)
(x_4 = 173 m², y_4 = 183,000 \$)
(x_5 = 210 m², y_5 = 223,000 \$)
⋮

$\boldsymbol{x}, \boldsymbol{y}$

Model



$\hat{\boldsymbol{y}}$

$\boldsymbol{y}$

Optimization method

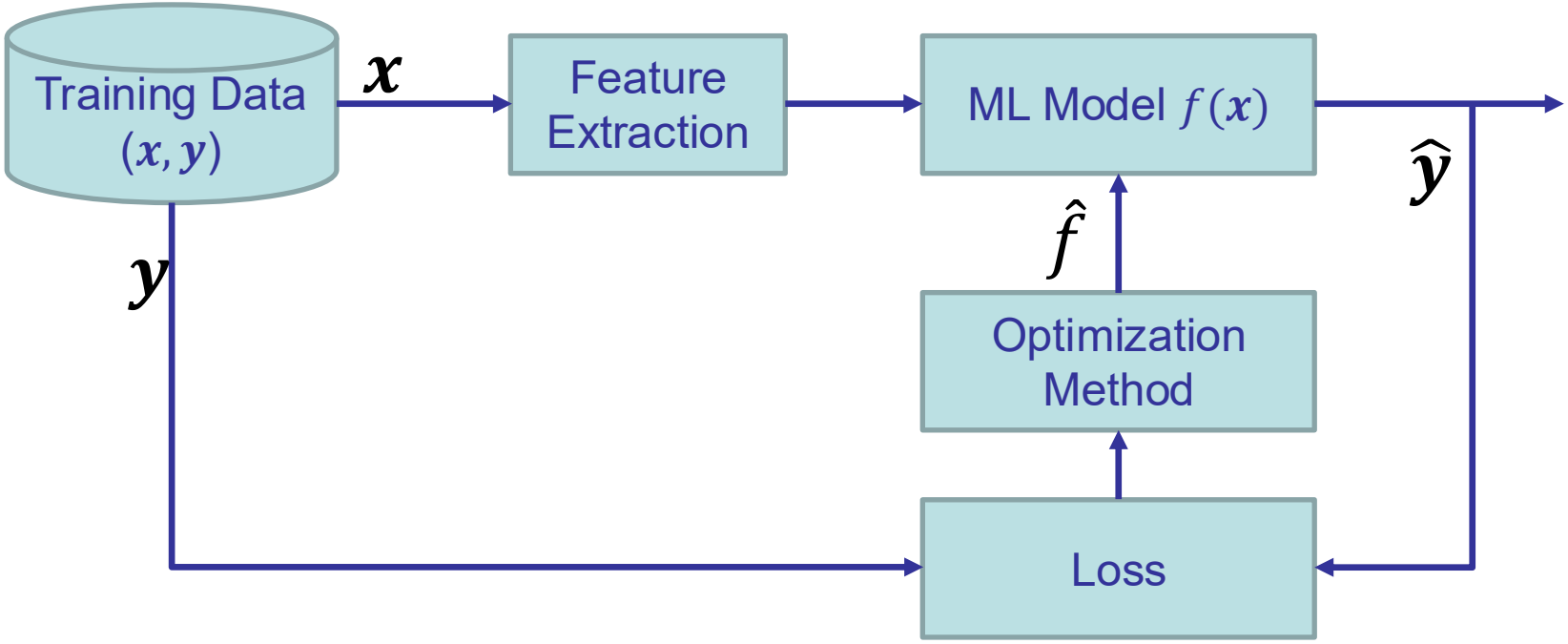
Gradient Descend to minimize SSE

Loss

Sum-Squared Error:

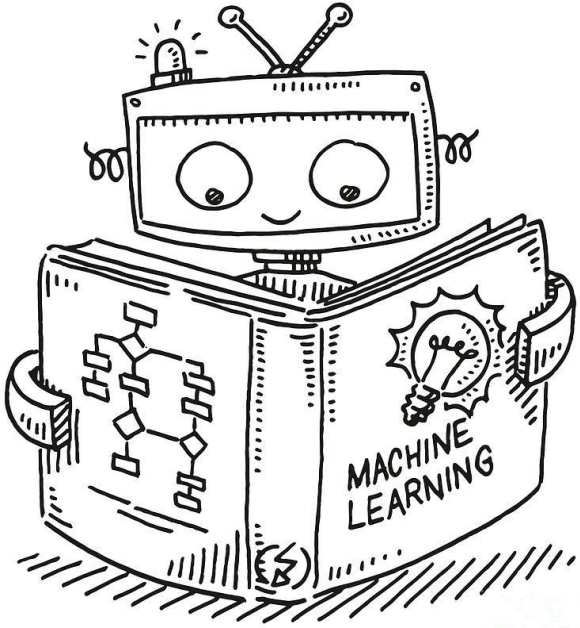
$$L(w_0, w_1) = \sum_{i=1}^{n} \left(y_i - (w_0 + w_1 x_i)\right)^2$$

Machine Learning Framework



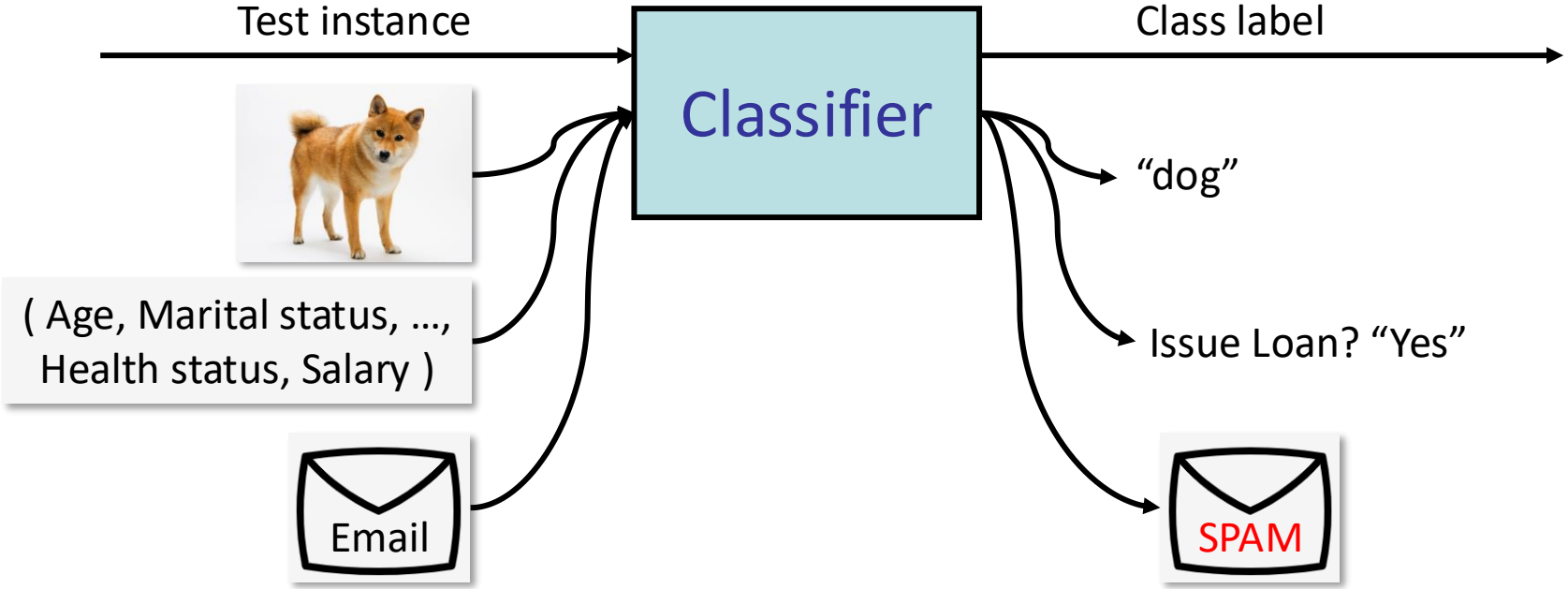
Today

- Regression: Linear Regression
- **Classification: Logistic Regression**
- Discussion



What is classification?

- What is classification?
  - identifying the class to which an instance belongs



What is classification?

- What is classification?
  - Learning a classifier

Training Phase

Training set

dog

cat

dog

cat

(30 years old, married, …, 3k $ per month) → loan issued

(20 years old, single, …, 1k $ per month) → loan reject

(40 years old, married, …, 5k $ per month) → loan issued

Testing Phase

Testing set

→ ?

(25 years old, single, …, 2k $ per month) → ?

Logistic Regression Classification

Input: **x =** “Easily best sushi in Seattle."

Sentiment Classifier

Output:

$\hat{y} = +1$

$\hat{y} = -1$

Linear Classifier

- Will use training data to learn a weight or coefficient for each word

Word	Coefficient
good	1.0
great	1.5
awesome	2.7
bad	-1.0
terrible	-2.1
awful	-3.3
restaurant, the, we, where, …	0.0
…	…

Linear Classifier

- Scoring a sentence

Word	Coefficient
…	…

Input $\mathbf{x}_i$:

Sushi was great, the food was awesome, but the service was terrible.

Simple linear classifier

Score(x) = weighted count of words in sentence

If *Score (x) > 0*:

$\hat{y} = +1$

Else

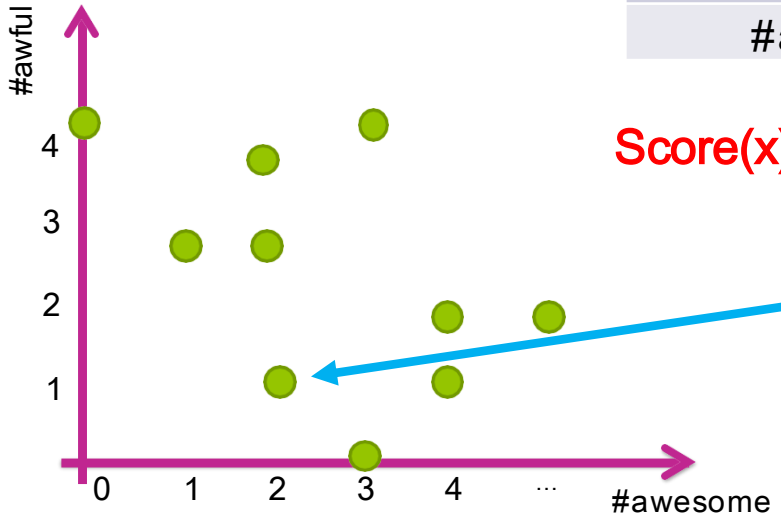
$\hat{y} = -1$

Called a linear classifier, because output is weighted sum of input.

Decision boundaries

- Suppose only two words had non-zero coefficient

Word	Coefficient
#awesome	1.0
#awful	-1.5



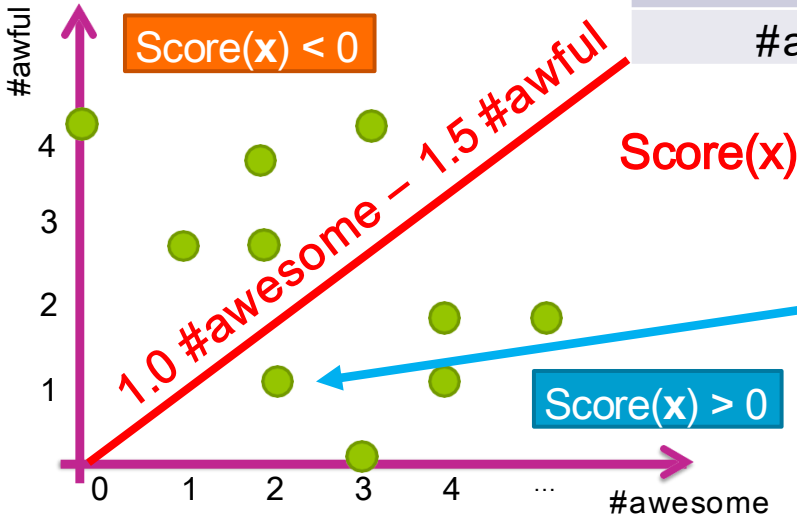
Score(x) = 1.0 #awesome – 1.5 #awful

Sushi was <u>awesome</u>,
the food was <u>awesome</u>,
but the service was <u>awful</u>.

Decision boundaries

- Suppose only two words had non-zero coefficient

Word	Coefficient
#awesome	1.0
#awful	-1.5

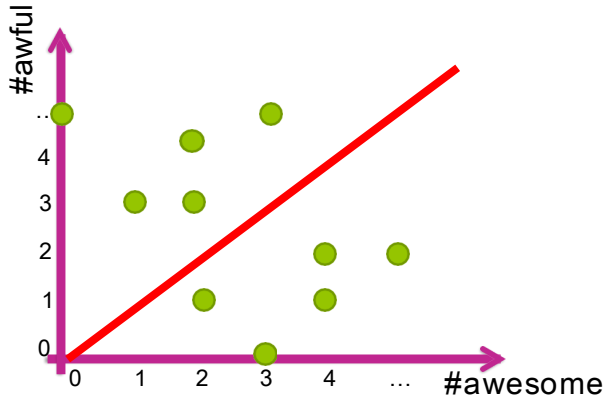


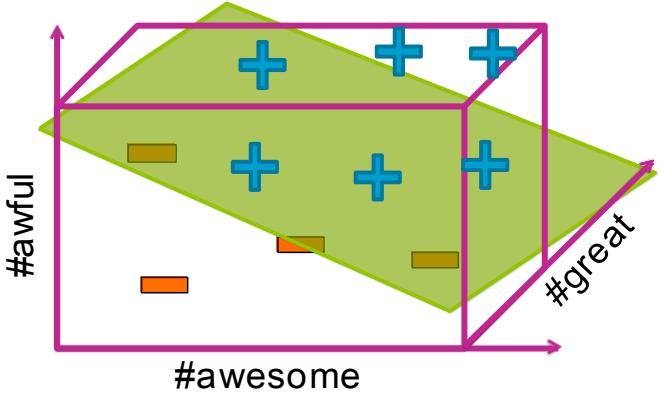
Score(x) = 1.0 #awesome – 1.5 #awful

Sushi was <u>awesome</u>,
the food was <u>awesome</u>,
but the service was <u>awful</u>.

Decision boundary separates positive & negative predictions

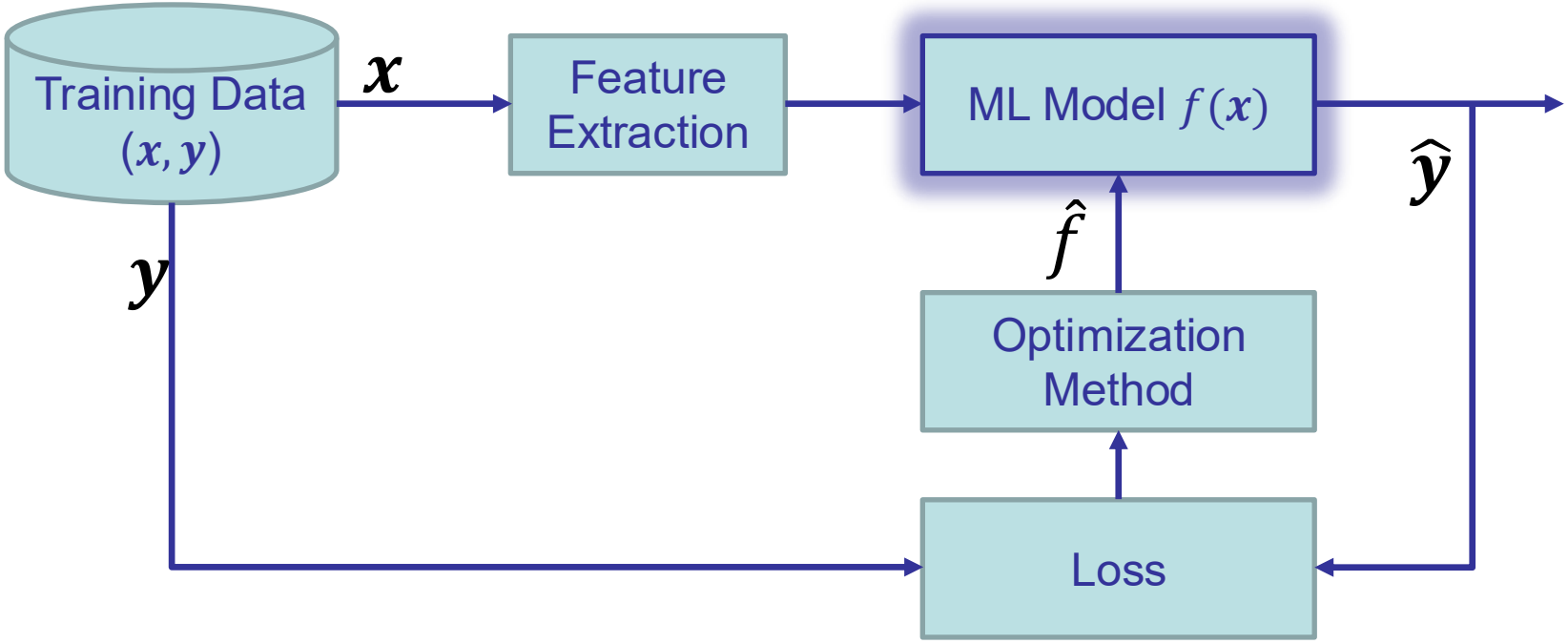
- For linear classifiers:
  - When 2 coefficients are non-zero → **line**
  - When 3 coefficients are non-zero → **plane**
  - When many coefficients are non-zero → **hyperplane**
  - For more general classifiers → **more complicated shapes**





Score(x) = w_0
+ w_1 #awesome
+ w_2 #awful
+ w_3 #great

Machine Learning Framework



Simple hyperplane

	Label	Documents
Training	-	Just plain boring
	-	Entirely predictable and lacks energy
	-	No surprises and very few laughs
	+	Very powerful
	+	The most fun film of the summer
Testing	?	Predictable with no fun

Model: $\hat{y}_i = \text{sgn}(\text{Score}(x_i))$

$$\text{Score}(\boldsymbol{x}_i) = w_0 + w_1 x_{i,1} + w_2 x_{i,2} \cdots + w_d x_{i,d} = \boldsymbol{w}^T \boldsymbol{x}_i$$

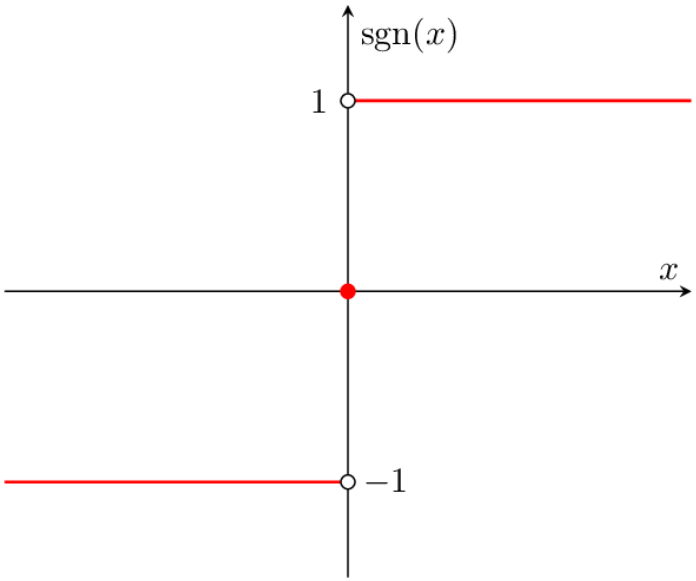
feature 1 = 1

feature 2 = $x_{i,1}$ e.g., #awesome

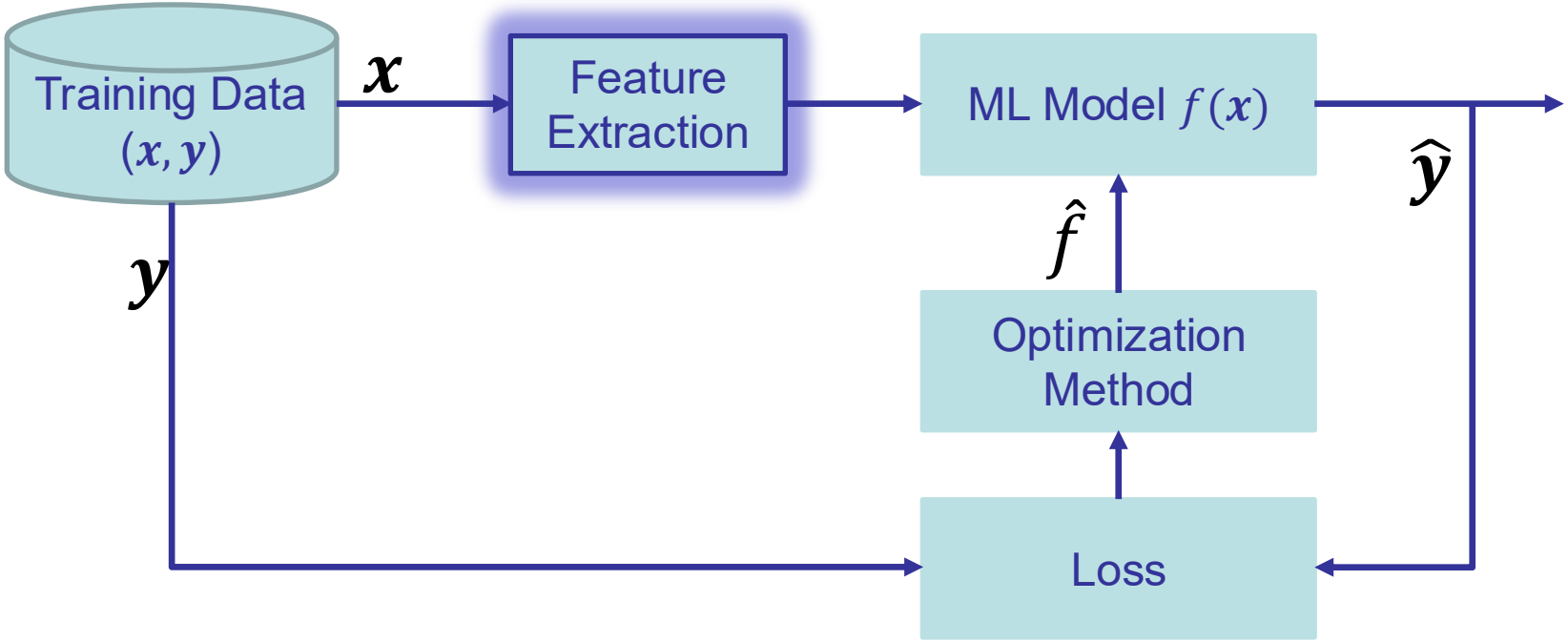
feature 3 = $x_{i,2}$ e.g., #awful

…

feature d+1 = $x_{i,d}$ e.g., #noodles



Machine Learning Framework



More generic features… D-dimensional hyperplane

Model: $\hat{y}_i = \text{sign}(\text{Score}(x_i))$

$$\text{Score}(\boldsymbol{x}_i) = w_0 + w_1 h_0(\boldsymbol{x}_i) + w_2 h_2(\boldsymbol{x}_i) \cdots + w_d h_d(\boldsymbol{x}_i) = \boldsymbol{w}^T h(\boldsymbol{x}_i)$$

feature 0 = $h_0(\mathbf{x})$ … e.g., 1

feature 1 = $h_1(\mathbf{x})$ … e.g., = #awesome

feature 2 = $h_2(\mathbf{x})$ … e.g., = #awful

or, log(#awful)

or, log(#bad) x #awful

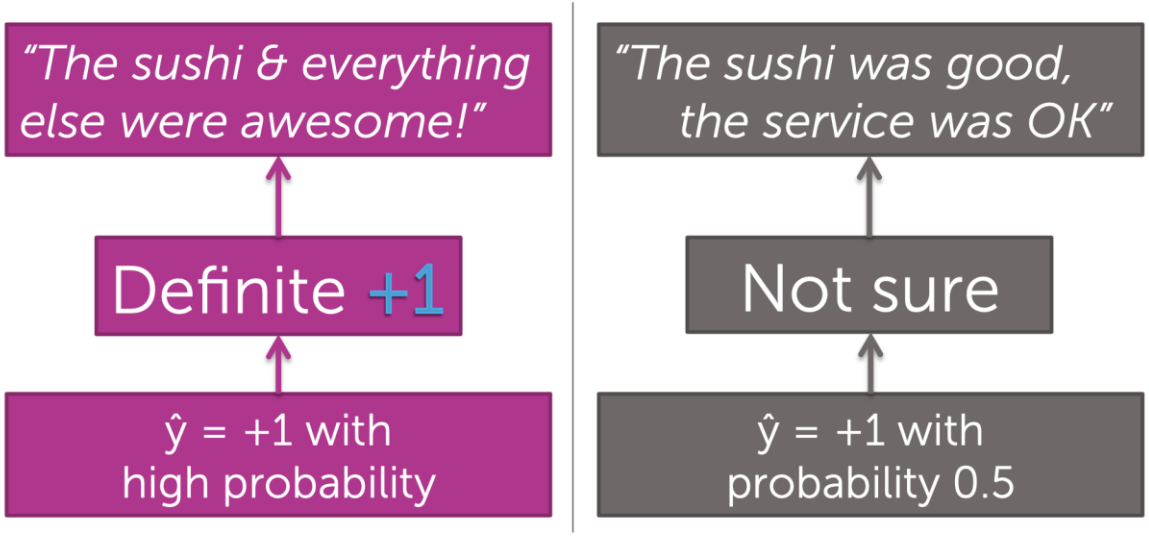
or, tf-idf("awful")

…

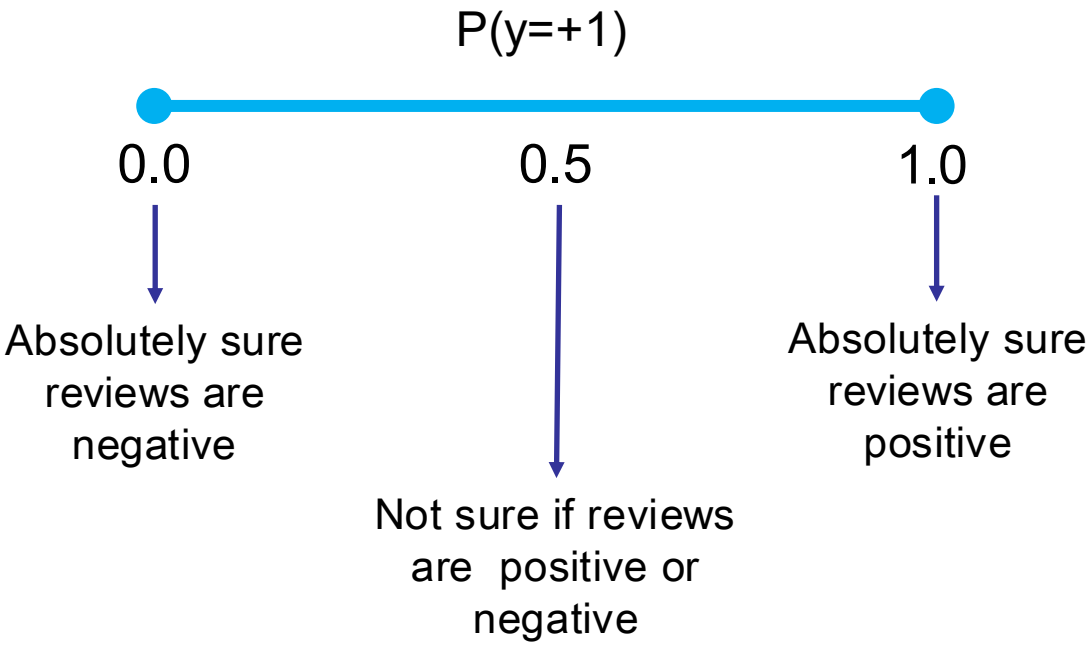
feature d = $h_d(\mathbf{x})$ …some other function

How confident is your prediction?

- Thus far, we've outputted a prediction **-1** or **+1**
- But, how sure are you about the prediction?



Interpreting probabilities as degrees of belief



Interpreting conditional probabilities

P(y=+1 | x = "All the sushi was delicious")

Absolutely sure reviews are negative

Not sure if reviews are positive or negative

Absolutely sure reviews are positive

How confident is your prediction?

"The sushi & everything else were awesome!"

Definite +1

$P(y=+1|\mathbf{x}=$ *"The sushi & everything else were awesome!"* $)$
$= 0.99$

"The sushi was good, the service was OK"

Not sure

$P(y=+1|\mathbf{x}=$ *"The sushi was good, the service was OK"* $)$
$= 0.55$

Many classifiers provide a degree of certainty:

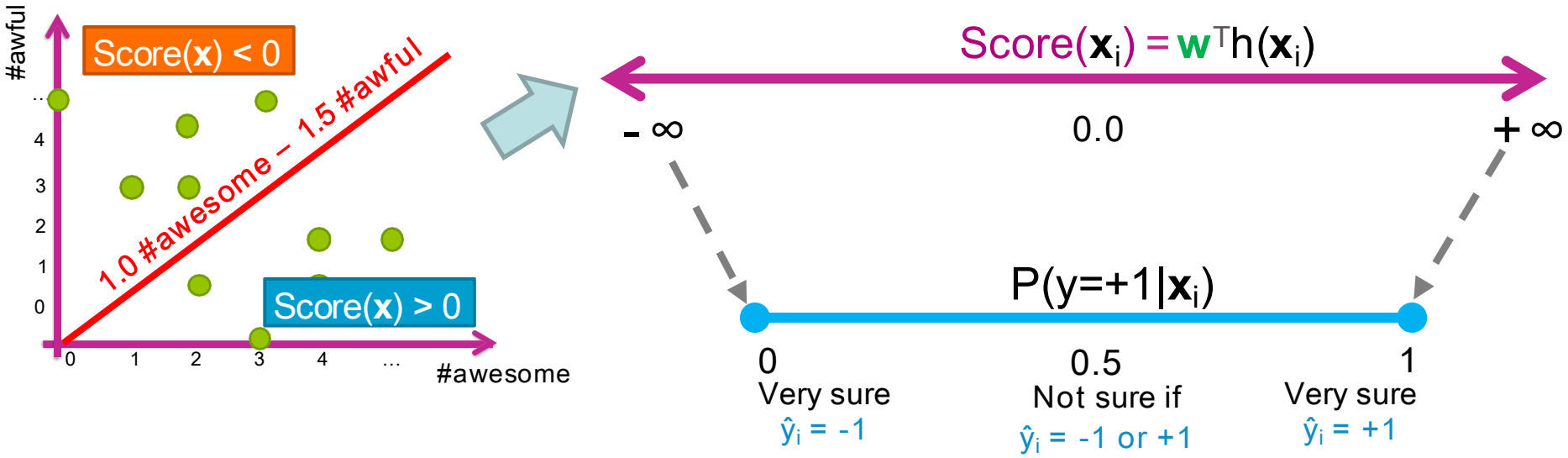
Output label

Input sentence

$$P(y|\mathbf{x})$$

Extremely useful in practice

Predicting class probabilities with generalized linear models

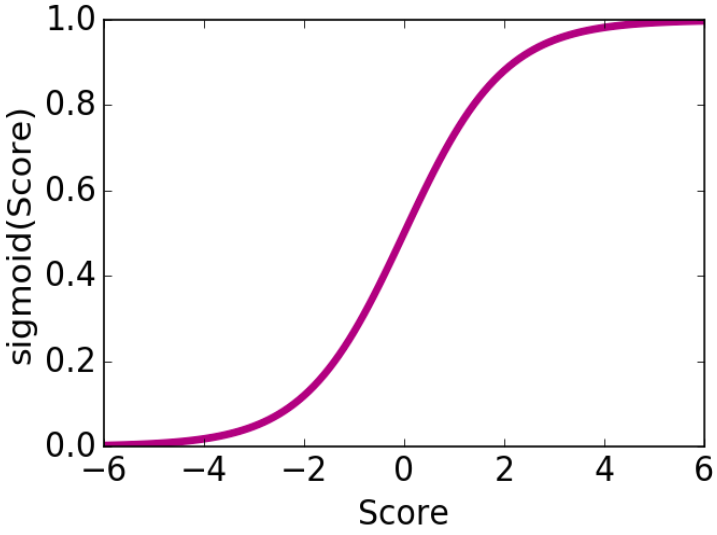


Generalized linear model : $P(y = +1|\boldsymbol{x}_i) = g(\boldsymbol{w}^T h(\boldsymbol{x}_i))$

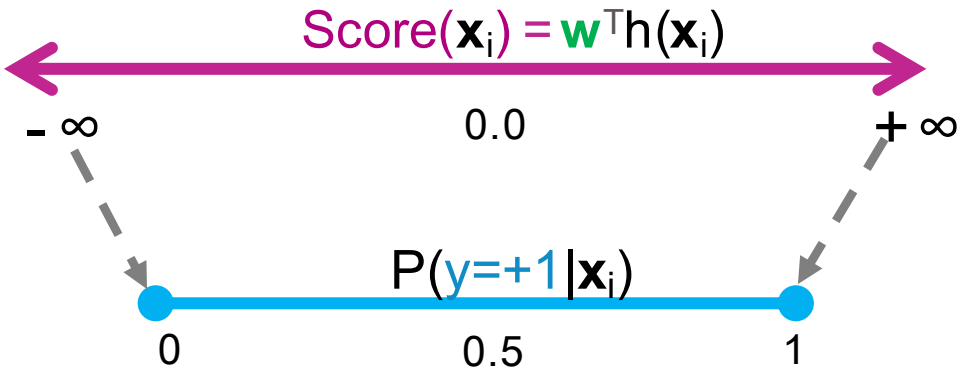
Logistic regression classifier

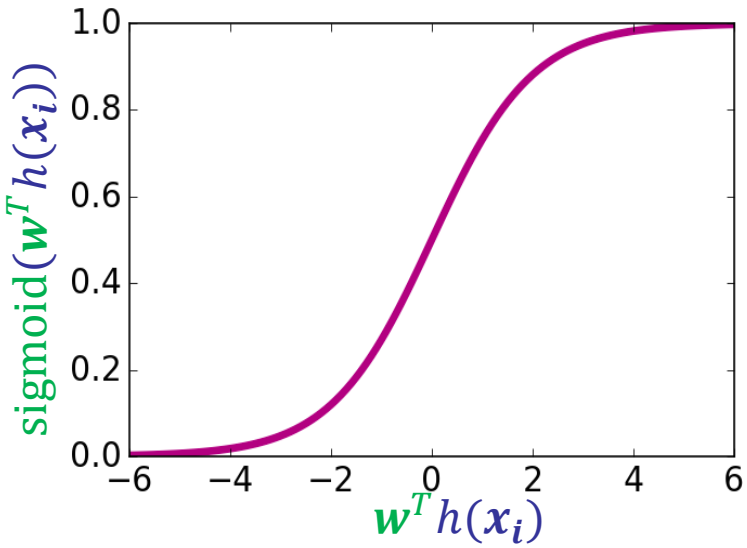
$$\text{Sigmoid}(\text{Score}) = \frac{1}{1+e^{-\text{Score}}}$$

Score	- ∞	-2	0	2	+∞
sigmoid(Score)	0	0.12	0.5	0.88	1



Logistic regression model





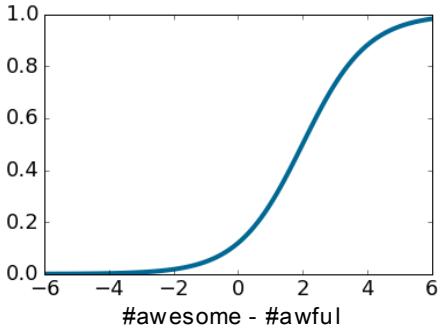
$$P(y = +1|\boldsymbol{x}_i, \boldsymbol{w}) \;=\; \text{sigmoid}(\boldsymbol{w}^T h(\boldsymbol{x}_i))$$

$$P(y = +1|\boldsymbol{x}_i, \boldsymbol{w}) \;= \sigma(\boldsymbol{w}^T h(\boldsymbol{x}_i))$$

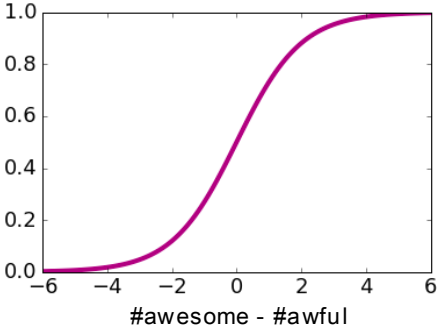
Effect of coefficients on logistic regression model

$$P(y = +1|\boldsymbol{x}_i, \boldsymbol{w}) = \sigma(\boldsymbol{w}^T h(\boldsymbol{x}_i))$$

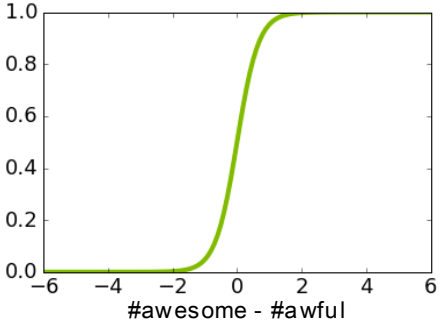
w_0	-2
$w_{\#awesome}$	+1
$w_{\#awful}$	-1



w_0	0
$w_{\#awesome}$	+1
$w_{\#awful}$	-1



w_0	0
$w_{\#awesome}$	+3
$w_{\#awful}$	-3



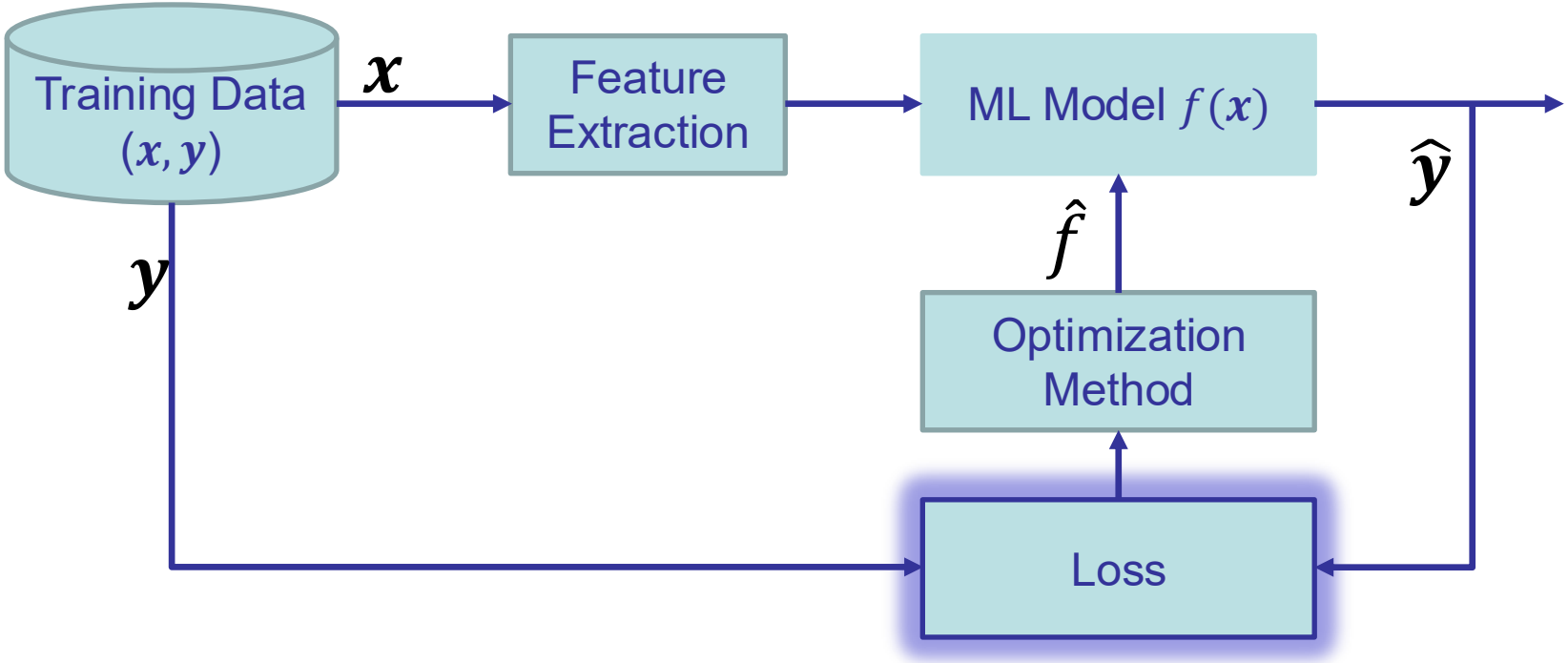
Learning the coefficient

x^1= #awesome	x^2 = #awful	y = sentiment
2	1	+1
0	2	-1
3	3	-1
4	1	+1
1	1	+1
2	4	-1
0	3	-1
0	1	-1
2	1	+1

Optimize Loss on training data

$\hat{w}$

Machine Learning Framework



Loss function

- Loss function or cost function
  - A measure of distance between classifier prediction and true label for a given set of parameters

$$L(p, y) = \text{how much } p \text{ differs from true } y$$

$$P(y = +1 | \boldsymbol{x}_i, \boldsymbol{w}) = \text{sigmoid}(\boldsymbol{w}^T h(\boldsymbol{x}_i))$$

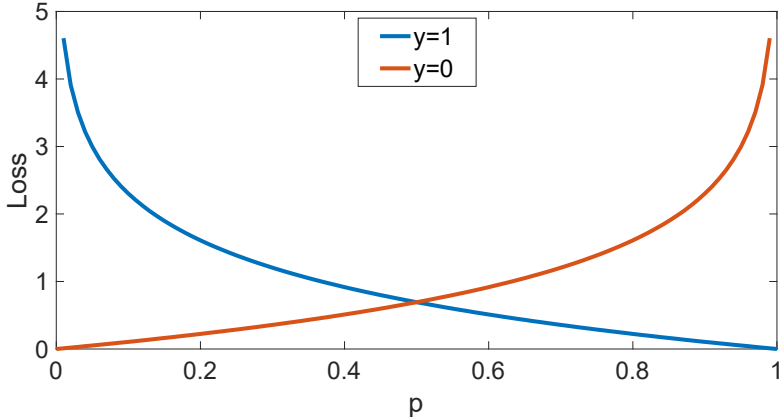
{1, 0}

The cross-entropy loss function

- Loss function used for logistic regression and often for neural networks

$$L(p, y) = \text{how much } p \text{ differs from true } y$$

$$= -y \log p - (1 - y) \log(1 - p)$$



The cross-entropy loss function

- Loss function used for logistic regression and often for neural networks

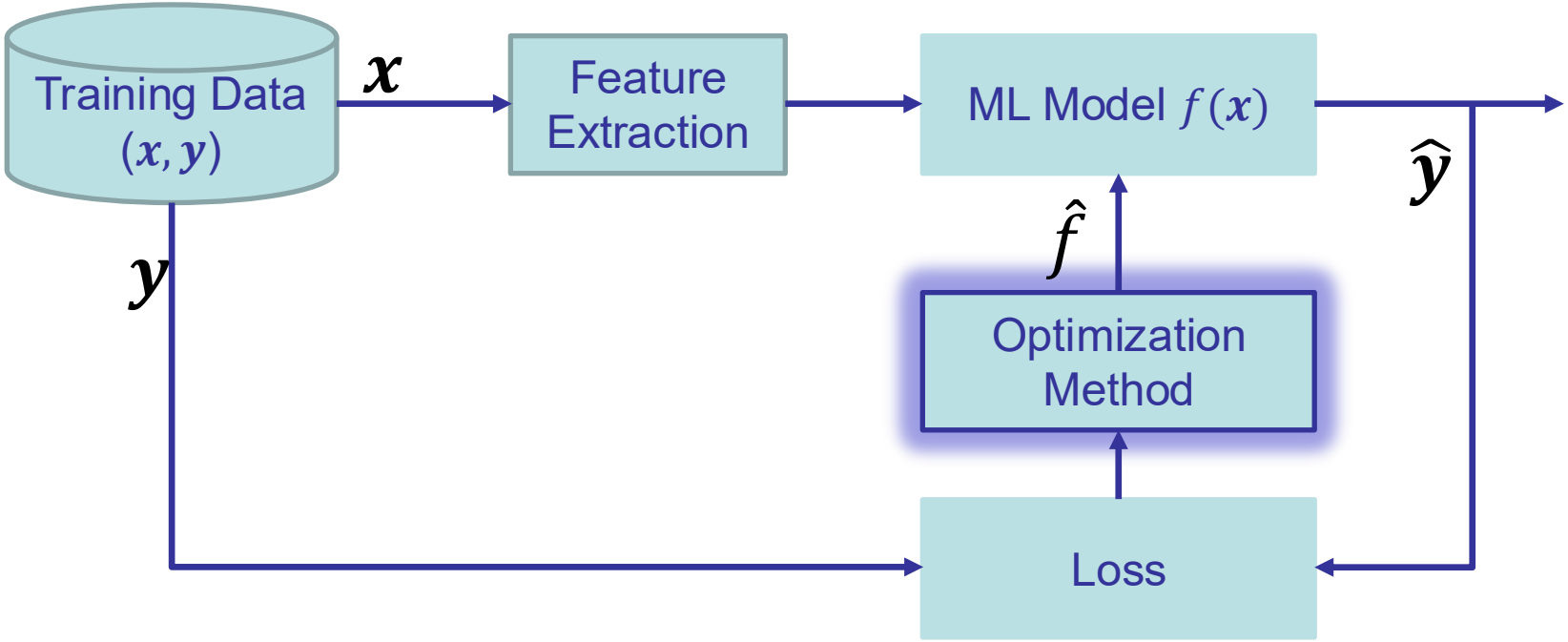
$$L(p, y) = \text{how much } p \text{ differs from true } y$$

$$= -y \log p - (1 - y) \log(1 - p)$$

$$= -y_i \log \sigma(\boldsymbol{w}^T h(\boldsymbol{x_i})) - (1 - y_i) \log(1 - \sigma(\boldsymbol{w}^T h(\boldsymbol{x_i}))\ )$$

$$\sum_{i=1}^{n} -y_i \log \sigma(\boldsymbol{w}^T h(\boldsymbol{x_i})) - (1 - y_i) \log(1 - \sigma(\boldsymbol{w}^T h(\boldsymbol{x_i}))\ )$$

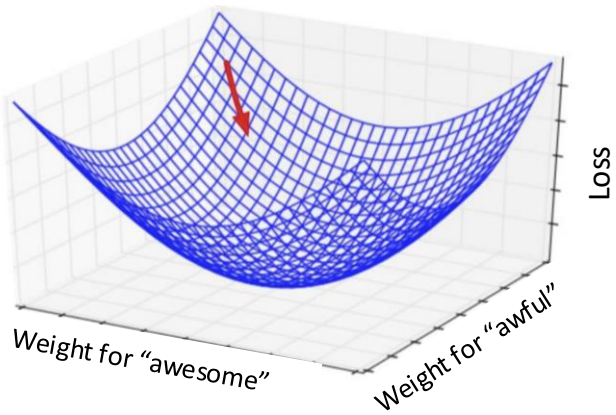
Machine Learning Framework



Gradient descent

To find the best $\boldsymbol{w}$ that minimize:

$$L = \sum_{i=1}^{n} -y_i \log \sigma(\boldsymbol{w}^T h(\boldsymbol{x_i})) - (1 - y_i) \log(1 - \sigma(\boldsymbol{w}^T h(\boldsymbol{x_i})) \ )$$



While not converged:

$$\boldsymbol{w}^{(t+1)} \leftarrow \boldsymbol{w}^{(t)} - \eta \frac{dL(\boldsymbol{w})}{d\boldsymbol{w}}$$

$$t \leftarrow t + 1$$

$$\frac{dL(\boldsymbol{w})}{d(\boldsymbol{w})} = \begin{bmatrix} \frac{\partial L(\boldsymbol{w})}{\partial w_1} \\ \frac{\partial L(\boldsymbol{w})}{\partial w_2} \\ \frac{\partial L(\boldsymbol{w})}{\partial w_3} \\ \vdots \end{bmatrix}$$

Gradient descent

To find the best $\boldsymbol{w}$ that minimize:

$$L = \sum_{i=1}^{n} -y_i \log \sigma(\boldsymbol{w}^T h(\boldsymbol{x_i})) - (1 - y_i) \log(1 - \sigma(\boldsymbol{w}^T h(\boldsymbol{x_i})) \ )$$

$$\frac{\partial L(\boldsymbol{w})}{\partial w_k} = \sum_{i=1}^{n} \frac{\partial}{\partial p_i}(-y_i \log p_i - (1 - y_i) \log(1 - p_i)) \frac{\partial p_i}{\partial w_k}$$

$$\frac{\partial p_i}{\partial z_i} \quad \frac{\partial z_i}{\partial w_k}$$

$$\frac{dL(\boldsymbol{w})}{d(\boldsymbol{w})} = \begin{bmatrix} \frac{\partial L(\boldsymbol{w})}{\partial w_1} \\ \frac{\partial L(\boldsymbol{w})}{\partial w_2} \\ \frac{\partial L(\boldsymbol{w})}{\partial w_3} \\ \vdots \end{bmatrix}$$

$$p_i = \sigma(\boldsymbol{w}^T h(\boldsymbol{x_i}))$$

$$z_i = \boldsymbol{w}^T h(\boldsymbol{x_i})$$

$$\frac{\partial p_i}{\partial z_i} = \frac{\partial}{\partial z_i}\left(\frac{1}{1 + e^{-z_i}}\right)$$

$$= -\frac{1}{(1 + e^{-z_i})^2} (e^{-z_i})(-1)$$

$$= p_i(1 - p_i)$$

$$= \sum_{i=1}^{n} \left(-\frac{y_i}{p_i} + \frac{1 - y_i}{1 - p_i} \right) \ p_i(1 - p_i) \ h(\boldsymbol{x_i})$$

$$= \sum_{i=1}^{n} (-y_i(1 - p_i) + (1 - y_i)p_i \ ) \ h(x_i)$$

$$= \sum_{i=1}^{n} (\sigma(\boldsymbol{w}^T h(\boldsymbol{x_i})) - y_i) \ h(x_i)$$

Gradient descent

To find the best $\boldsymbol{w}$ that minimize:

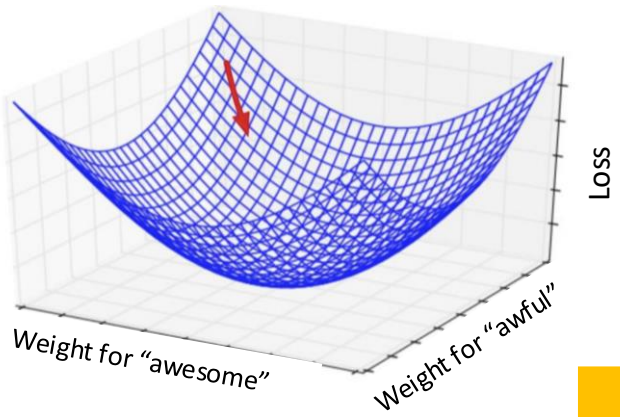
$$L = \sum_{i=1}^{n} -y_i \log \sigma(\boldsymbol{w}^T h(\boldsymbol{x_i})) - (1 - y_i) \log(1 - \sigma(\boldsymbol{w}^T h(\boldsymbol{x_i}))\ )$$

While not converged:

$$\boldsymbol{w}^{(t+1)} \leftarrow \boldsymbol{w}^{(t)} - \eta \frac{dL(\boldsymbol{w})}{d\boldsymbol{w}}$$

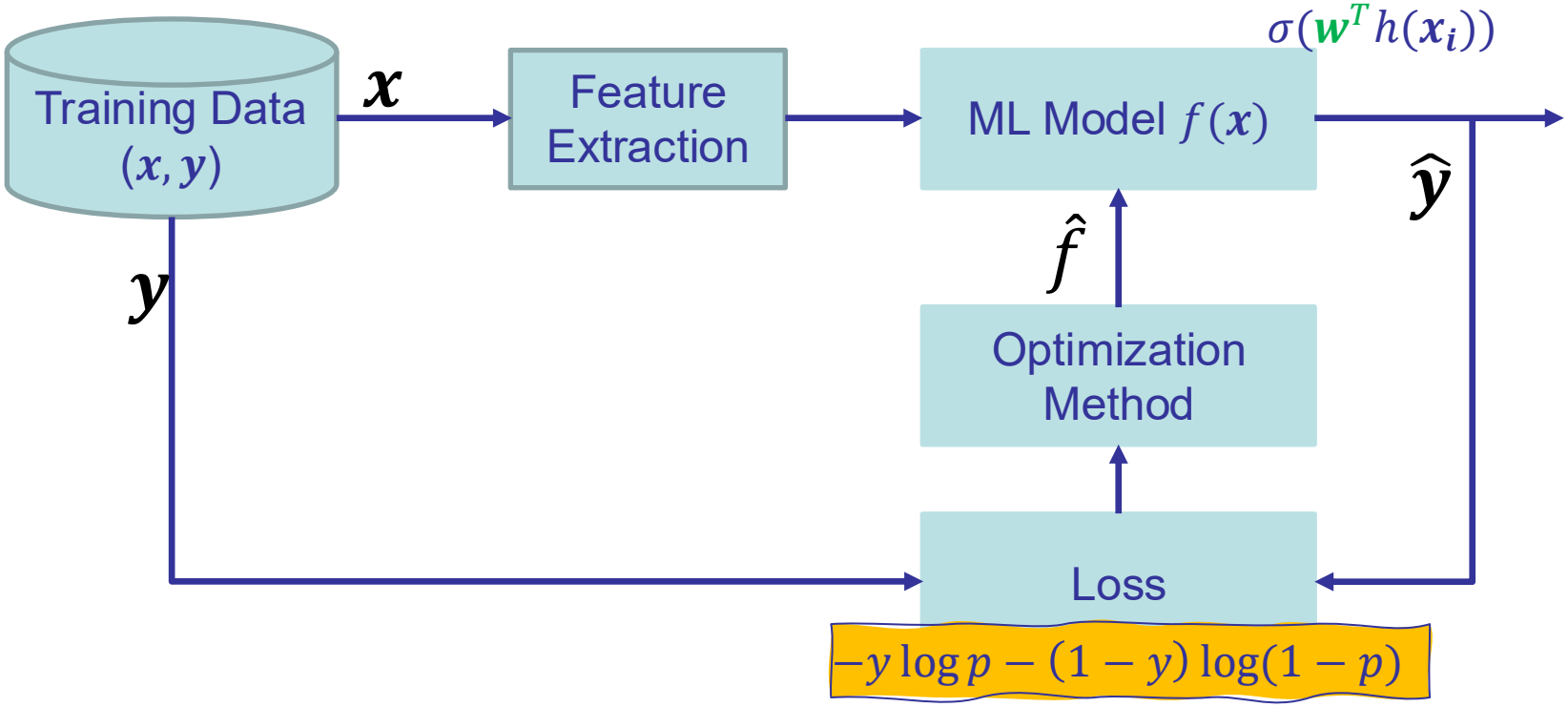
$$t \leftarrow t + 1$$

$$\frac{dL(\boldsymbol{w})}{d(\boldsymbol{w})} = \begin{bmatrix} \frac{\partial L(\boldsymbol{w})}{\partial w_1} \\ \frac{\partial L(\boldsymbol{w})}{\partial w_2} \\ \frac{\partial L(\boldsymbol{w})}{\partial w_3} \\ \vdots \end{bmatrix}$$



$$\frac{\partial L(\boldsymbol{w})}{\partial w_k} = \sum_{i=1}^{n} \left(\sigma\left(\boldsymbol{w}^T\left(h(\boldsymbol{x}_i)\right)\right) - y_i\right) h_k(\boldsymbol{x}_i)$$

Difference between the model prediction and the correct answer y

Feature value for dimension *k*

Recap: Logistic Regression Classifier